

```
!pip install -q --upgrade scipy
!pip install -q monai nibabel timm tqdm scikit-learn
import numpy, scipy, monai, timm
print("All OK – restart runtime now")
```

```

_____ 62.3/62.3 kB 1.3 MB/s eta 0:00:00
_____ 35.3/35.3 MB 44.6 MB/s eta 0:00:00
_____ 1.6/1.6 MB 20.5 MB/s eta 0:00:00

All OK – restart runtime now
```

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
Mounted at /content/drive
```

```
# Install synapse client
!pip install -q synapseclient

import synapseclient
import os

# Mount Drive if not already mounted
from google.colab import drive
drive.mount('/content/drive')

# Create destination folder on Drive
BRATS_DIR = '/content/drive/MyDrive/BraTS2023-GLI'
os.makedirs(BRATS_DIR, exist_ok=True)

# Login with your Synapse auth token
# Get token from: synapse.org → your profile → Access Tokens
syn = synapseclient.Synapse()
syn.login(authToken="eyJ0eXAiOiJKV1QiLCJraWQiOiJXN0500ldMSlQ6SjVSSzpmN1RM0lQ3TDc6M1ZYNjpkRU9V0jY0NFI6VTNJWDo1S1oy0jdaQ0s6RlB")

# Download directly to Drive
print("Starting download to Drive...")
entity = syn.get(
    entity='syn51514132',
    downloadLocation=BRATS_DIR
```

)

```
print(f"Downloaded to: {entity.path}")
print(f"File: {os.path.basename(entity.path)}")
```

```

_____ 836.8/836.8 kB 6.6 MB/s eta 0:00:00
_____ 155.6/155.6 kB 12.1 MB/s eta 0:00:00
_____ 73.1/73.1 kB 4.7 MB/s eta 0:00:00
_____ 1.0/1.0 MB 25.3 MB/s eta 0:00:00
_____ 131.1/131.1 kB 10.8 MB/s eta 0:00:00
_____ 88.0/88.0 kB 5.7 MB/s eta 0:00:00

```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is ipython 7.34.0 requires jedi>=0.16, which is not installed.

google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.34.2 which is incompatible.

torch 2.11.0+cpu requires setuptools<82, but you have setuptools 82.0.1 which is incompatible.

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
Welcome, aryaanswain!

INFO:synapseclient_default:Welcome, aryaanswain!

[WARNING] /tmp/ipykernel_1356/4170708504.py:22: DeprecationWarning: Call to deprecated method get. (To be removed in 5.0.0. U
entity = syn.get(

WARNING:py.warnings:/tmp/ipykernel_1356/4170708504.py:22: DeprecationWarning: Call to deprecated method get. (To be removed i
entity = syn.get(

Starting download to Drive...

Downloading files: 100%|██████████| 13.2G/13.2G [05:18<00:00, 41.4MB/s, syn51514132][syn51514132]: Downloaded to /content/dri
INFO:synapseclient_default:[syn51514132]: Downloaded to /content/drive/MyDrive/BraTS2023-GLI/ASNR-MICCAI-BraTS2023-GLI-Challe
Downloading files: 100%|██████████| 13.2G/13.2G [05:18<00:00, 41.3MB/s, syn51514132]Downloaded to: /content/drive/MyDrive/Bra
File: ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData.zip

```
import zipfile, time, os
```

```
zip_path = '/content/drive/MyDrive/BraTS2023-GLI/ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData.zip'
extract_dir = '/content/drive/MyDrive/BraTS2023-GLI'
```

```
print(f"Zip size: {os.path.getsize(zip_path)/1e9:.2f} GB")
print("Extracting...")
start = time.time()
with zipfile.ZipFile(zip_path, 'r') as zf:
    zf.extractall(extract_dir)
```

```
print(f"Done in {time.time()-start:.1f}s")
```

```
# Check what was extracted
```

```
contents = [f for f in os.listdir(extract_dir) if not f.endswith('.zip')]
```

```
print(f"\nExtracted folders/files: {len(contents)}")
```

```
print(f"First few: {contents[:3]}")
```

```
Zip size: 13.17 GB
```

```
Extracting...
```

```
Done in 451.7s
```

```
Extracted folders/files: 1
```

```
First few: ['ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData']
```

```
import nibabel as nib
```

```
import numpy as np
```

```
# Find first patient folder
```

```
brats_data_dir = '/content/drive/MyDrive/BraTS2023-GLI'
```

```
# The extracted folder might be nested one level deep
```

```
contents = os.listdir(brats_data_dir)
```

```
print("Top level contents:", contents[:5])
```

```
# Find a patient folder starting with BraTS-GLI
```

```
patient_folders = [f for f in os.listdir(brats_data_dir)
                    if f.startswith('BraTS-GLI')]
```

```
if not patient_folders:
```

```
    # Check one level deeper
```

```
    for item in contents:
```

```
        subpath = os.path.join(brats_data_dir, item)
```

```
        if os.path.isdir(subpath):
```

```
            sub = os.listdir(subpath)
```

```
            brats_in_sub = [f for f in sub if f.startswith('BraTS-GLI')]
```

```
            if brats_in_sub:
```

```
                print(f"Patients are inside subfolder: {item}/")
```

```
                print(f"Sample patient: {brats_in_sub[0]}")
```

```
                print(f"Files: {os.listdir(os.path.join(subpath, brats_in_sub[0]))}")
```

```
                # Check label values
```

```
                seg = nib.load(os.path.join(subpath, brats_in_sub[0],
```

```
                                f'{brats_in_sub[0]}-seg.nii.gz')).get_fdata()
```

```
                print(f"\nLabel values: {np.unique(seg)}")
```

```
                break
```

```

else:
    print(f"Found {len(patient_folders)} patient folders directly")
    sample = patient_folders[0]
    print(f"Sample: {sample}")
    print(f"Files: {os.listdir(os.path.join(brats_data_dir, sample))}")
    seg = nib.load(os.path.join(brats_data_dir, sample,
                                f'{sample}-seg.nii.gz')).get_fdata()
    print(f"\nLabel values: {np.unique(seg)}")

```

Top level contents: ['ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData.zip', 'ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData/']
 Patients are inside subfolder: ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData/
 Sample patient: BraTS-GLI-00324-000
 Files: ['BraTS-GLI-00324-000-t1c.nii.gz', 'BraTS-GLI-00324-000-t2w.nii.gz', 'BraTS-GLI-00324-000-t1n.nii.gz', 'BraTS-GLI-00324-000-t2f.nii.gz']
 Label values: [0. 1. 2. 3.]

```

from google.colab import drive
drive.mount('/content/drive')

import os, torch

# — Check checkpoint —————
FINETUNE_CKPT = '/content/drive/MyDrive/checkpoints/swinunetr_ucsf_finetuned.pt'

if os.path.exists(FINETUNE_CKPT):
    ckpt = torch.load(FINETUNE_CKPT, map_location='cpu', weights_only=False)
    print(f"✓ Checkpoint found")
    print(f"  Epoch:      {ckpt['epoch']}")
    print(f"  Val Dice:    {ckpt['val_dice']:.4f}")
else:
    print(f"✗ Checkpoint NOT found at {FINETUNE_CKPT}")

# — Check UCSF dataset —————
UCSF_DIR = '/content/drive/MyDrive/UCSF-PDGM-v5'
UCSF_CSV = '/content/drive/MyDrive/UCSF-PDGM-metadata_v5.csv'

if os.path.exists(UCSF_DIR):
    folders = [f for f in os.listdir(UCSF_DIR) if os.path.isdir(os.path.join(UCSF_DIR, f))]
    print(f"\n✓ UCSF dataset found")
    print(f"  Patient folders: {len(folders)}")
else:
    print(f"\n✗ UCSF dataset NOT found at {UCSF_DIR}")

```

```

if os.path.exists(UCSF_CSV):
    import pandas as pd
    df = pd.read_csv(UCSF_CSV)
    print(f"✓ Metadata CSV found – {len(df)} rows")
else:
    print(f"✗ Metadata CSV NOT found at {UCSF_CSV}")

# — Check feature cache —————
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'

if os.path.exists(CACHE_DIR):
    files = [f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')]
    print(f"\n✓ Feature cache found")
    print(f"  Files: {len(files)}")
else:
    print(f"\n✗ Feature cache NOT found at {CACHE_DIR}")

# — Write test —————
test_path = '/content/drive/MyDrive/checkpoints/write_test.txt'
try:
    with open(test_path, 'w') as f:
        f.write('test')
    os.remove(test_path)
    print(f"\n✓ Drive is writable")
except Exception as e:
    print(f"\n✗ Drive is READ-ONLY: {e}")

```

```

Mounted at /content/drive
✓ Checkpoint found
  Epoch:    97
  Val Dice: 0.8846

✓ UCSF dataset found
  Patient folders: 501
✓ Metadata CSV found – 501 rows

✓ Feature cache found
  Files: 487

✓ Drive is writable

```

```

# =====
# CELL 2 – Mount Drive + Config
# =====

from google.colab import drive
drive.mount('/content/drive')

import os, torch

class Config:
    UCSF_DIR    = '/content/drive/MyDrive/UCSF-PDGM-v5'
    UCSF_CSV    = '/content/drive/MyDrive/UCSF-PDGM-metadata_v5.csv'
    CKPT_DIR    = '/content/drive/MyDrive/checkpoints'
    WEIGHTS     = '/content/drive/MyDrive/checkpoints/swinunetr_brats21_fold1.pt'

    SPATIAL_SIZE = (128, 128, 128)
    VOXEL_SPACING = (1.0, 1.0, 1.0)

    IN_CHANNELS  = 4
    OUT_CHANNELS = 3
    FEATURE_SIZE = 48
    ENCODER_DIM  = 768      # Stage 4 bottleneck channels

    TTA_RUNS = 5

    MORPH_FEATURES = 8
    FUSED_DIM      = 512
    SEG_THRESHOLD  = 0.3

    # — Ablation control —————
    # 'A' encoder only | 'B' fixed gate | 'C' G=conf | 'D' proposed
    GATE_MODE = 'D'

    # — Backbone control —————
    FREEZE_BACKBONE = True    # set False for fine-tune ablation

    BATCH_SIZE = 8            # bigger batch – features are tiny now
    LR          = 3e-4
    CLF_EPOCHS  = 60
    DEVICE      = 'cuda' if torch.cuda.is_available() else 'cpu'

cfg = Config()

```

```

os.makedirs(cfg.CKPT_DIR, exist_ok=True)
print(f"Device: {cfg.DEVICE}")
print(f"Gate mode: {cfg.GATE_MODE}  Backbone frozen: {cfg.FREEZE_BACKBONE}")
if cfg.DEVICE == 'cpu':
    print("WARNING: GPU not enabled")

def csv_id_to_file_id(csv_id):
    prefix, num = csv_id.rsplit('-', 1)
    return f"{prefix}-{num.zfill(4)}"

def get_patient_paths(csv_id, ucsf_dir):
    file_id = csv_id_to_file_id(csv_id)
    folder_path = os.path.join(ucsf_dir, f"{file_id}_nifti")
    return {
        'folder': folder_path,
        't1': os.path.join(folder_path, f'{file_id}_T1_bias.nii.gz'),
        't1c': os.path.join(folder_path, f'{file_id}_T1c_bias.nii.gz'),
        't2': os.path.join(folder_path, f'{file_id}_T2_bias.nii.gz'),
        'flair': os.path.join(folder_path, f'{file_id}_FLAIR_bias.nii.gz'),
        'label': os.path.join(folder_path, f'{file_id}_tumor_segmentation.nii.gz'),
    }

def verify_patient_files(csv_id, ucsf_dir):
    paths = get_patient_paths(csv_id, ucsf_dir)
    missing = [k for k, v in paths.items()
               if k != 'folder' and not os.path.exists(v)]
    return (len(missing) == 0), missing

print("Config and helpers loaded")

```

```

Mounted at /content/drive
Device: cpu
Gate mode: D  Backbone frozen: True
WARNING: GPU not enabled
Config and helpers loaded

```

```

# =====
# CELL 2B – Restore slim cache from Drive backup (fast session start)
# Run this every new session instead of rebuilding from full cache
# =====
import shutil, time, os

```

```

SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
SLIM_CACHE_LOCAL = '/content/feature_cache_slim_ucsf'

if not os.path.exists(SLIM_CACHE_LOCAL):
    if os.path.exists(SLIM_CACHE_DRIVE):
        print("Copying slim cache from Drive...")
        start = time.time()
        shutil.copytree(SLIM_CACHE_DRIVE, SLIM_CACHE_LOCAL)
        print(f"Done in {time.time()-start:.1f}s - "
              f"{len(os.listdir(SLIM_CACHE_LOCAL))} files")
    else:
        print("No Drive backup found – run Cell 11 + Cell 12 to build fresh")
else:
    print(f"Already local – {len(os.listdir(SLIM_CACHE_LOCAL))} files")

SLIM_CACHE = SLIM_CACHE_LOCAL
print(f"SLIM_CACHE = {SLIM_CACHE}")

```

```

Copying slim cache from Drive...
Done in 9.6s – 487 files
SLIM_CACHE = /content/feature_cache_slim_ucsf

```

```

# add zip of ucsf to local storage from drive
import shutil, zipfile, time, os

ZIP_PATH = '/content/drive/MyDrive/checkpoints/ucsf_essential.zip'
LOCAL_ZIP = '/content/ucsf_essential.zip'
LOCAL_DIR = '/content/UCSF-PDGM-v5'

print("Copying single zip file from Drive...")
start = time.time()
shutil.copy2(ZIP_PATH, LOCAL_ZIP)
print(f"Copy done in {time.time()-start:.1f}s")

print("Extracting...")
start = time.time()
with zipfile.ZipFile(LOCAL_ZIP, 'r') as zf:
    zf.extractall(LOCAL_DIR)
print(f"Extract done in {time.time()-start:.1f}s")

os.remove(LOCAL_ZIP)

```



```
cfg.UCSF_DIR = LOCAL_DIR
print(f"cfg.UCSF_DIR = {cfg.UCSF_DIR}")
```

```
Copying single zip file from Drive...
Copy done in 104.5s
Extracting...
Extract done in 56.6s
cfg.UCSF_DIR = /content/UCSF-PDGM-v5
```

```
print(cfg.UCSF_DIR)
print(os.path.exists(cfg.UCSF_DIR))
```

```
/content/UCSF-PDGM-v5
True
```

```
print(cfg.DEVICE)
print(next(swinunetr.parameters()).device if 'swinunetr' in dir() else "not built yet")
```

```
cuda
not built yet
```

```
# =====
# CELL 3 – Imports
# =====

import numpy as np
import pandas as pd
import torch.nn as nn
import torch.nn.functional as F
import nibabel as nib
from scipy import ndimage
from scipy.ndimage import distance_transform_edt, binary_erosion
from tqdm import tqdm
import monai

from monai.transforms import (
    Compose, LoadImage, EnsureChannelFirstd, ConcatItemsd,
    Orientationd, Spacingd, NormalizeIntensityd, CropForegroundd,
    ResizeWithPadOrCropd, RandFlipd, RandRotate90d,
    RandShiftIntensityd, ToTensord, MapLabelValued
)
```

```

from monai.networks.nets import SwinUNETR
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve, confusion_matrix,
    ConfusionMatrixDisplay
)

print(f"MONAI: {monai.__version__} Torch: {torch.__version__}")
print("All imports OK")

```

```

MONAI: 1.6.0 Torch: 2.11.0+cpu
All imports OK

```

```

# =====
# CELL 4 – UCSF data loading
# =====

def build_ucsf_dicts(csv_path, ucsf_dir):
    df = pd.read_csv(csv_path)
    data_dicts, missing_count = [], 0

    for _, row in df.iterrows():
        csv_id = row['ID']
        paths = get_patient_paths(csv_id, ucsf_dir)
        if not all(os.path.exists(paths[k])
                    for k in ['t1', 't1c', 't2', 'flair', 'label']):
            missing_count += 1
            continue

        idh_raw = str(row['IDH']).strip().lower()
        grade_raw = str(row['WHO CNS Grade']).strip()
        mgmt_raw = str(row['MGMT status']).strip().lower()

        idh = (0.0 if idh_raw == 'wildtype'
               else float('nan') if idh_raw in ['', 'nan', 'unknown']
               else 1.0)
        # Binary grade: low (2/3) = 0, high (4) = 1
        grade = (1.0 if grade_raw == '4'
                 else 0.0 if grade_raw in ['2', '3']
                 else float('nan'))

```

```

    mgmt = (1.0 if mgmt_raw == 'positive'
            else 0.0 if mgmt_raw == 'negative'
            else float('nan'))

    data_dicts.append({
        'patient_id': csv_id,
        't1': paths['t1'], 't1c': paths['t1c'],
        't2': paths['t2'], 'flair': paths['flair'],
        'label': paths['label'],
        'idh': idh, 'grade': grade, 'mgmt': mgmt,
    })

print(f"Patients loaded: {len(data_dicts)}")
print(f"Patients skipped: {missing_count}")
idh_0 = sum(1 for d in data_dicts if d['idh']==0.0)
idh_1 = sum(1 for d in data_dicts if d['idh']==1.0)
g_lo  = sum(1 for d in data_dicts if d['grade']==0.0)
g_hi  = sum(1 for d in data_dicts if d['grade']==1.0)
m_neg = sum(1 for d in data_dicts if d['mgmt']==0.0)
m_pos = sum(1 for d in data_dicts if d['mgmt']==1.0)
print(f"\nIDH: {idh_0} wt / {idh_1} mut")
print(f"Grade: {g_lo} low / {g_hi} high")
print(f"MGMT: {m_neg} neg / {m_pos} pos")
return data_dicts

def get_ucsf_transforms(mode='train'):
    base = [
        LoadImaged(keys=['t1', 't1c', 't2', 'flair', 'label']),
        EnsureChannelFirstd(keys=['t1', 't1c', 't2', 'flair', 'label']),
        ConcatItemsd(keys=['t1', 't1c', 't2', 'flair'], name='image'),
        Orientationd(keys=['image', 'label'], axcodes='RAS'),
        Spacingd(keys=['image', 'label'], pixdim=config.VOXEL_SPACING,
                 mode=('bilinear', 'nearest')),
        NormalizeIntensityd(keys='image', nonzero=True, channel_wise=True),
        CropForegroundd(keys=['image', 'label'], source_key='image'),
        ResizeWithPadOrCrop(keys=['image', 'label'],
                             spatial_size=config.SPATIAL_SIZE),
        MapLabelValued(keys=['label'], orig_labels=[0,1,2,4],
                       target_labels=[0,1,2,3]),
        ToTensord(keys=['image', 'label'])
    ]

```

```

aug = [
    RandFlipd(keys=['image','label'], prob=0.5, spatial_axis=0),
    RandFlipd(keys=['image','label'], prob=0.5, spatial_axis=1),
    RandFlipd(keys=['image','label'], prob=0.5, spatial_axis=2),
    RandRotate90d(keys=['image','label'], prob=0.5, max_k=3),
    RandShiftIntensityd(keys=['image'], offsets=0.1, prob=0.5),
]
return Compose(base[:-1]+aug+[base[-1]] if mode=='train' else base)

def stratified_split(all_dicts, val_split=0.15, test_split=0.15, seed=42):
    """
    Stratifies on a combined (IDH, Grade) key so both labels
    are proportionally represented in train/val/test.
    MGMT is not included in the key (would create too many
    small groups) but tends to follow along reasonably.
    """
    import random
    rng = random.Random(seed)

    def key(d):
        idh = 'x' if np.isnan(d['idh']) else str(int(d['idh']))
        grd = 'x' if np.isnan(d['grade']) else str(int(d['grade']))
        return (idh, grd)

    groups = {}
    for d in all_dicts:
        groups.setdefault(key(d), []).append(d)

    train_d, val_d, test_d = [], [], []
    for k, group in groups.items():
        shuffled = group[:]
        rng.shuffle(shuffled)
        n = len(shuffled)
        n_test = int(n * test_split)
        n_val = int(n * val_split)
        test_d += shuffled[:n_test]
        val_d += shuffled[n_test:n_test+n_val]
        train_d += shuffled[n_test+n_val:]

    return train_d, val_d, test_d

```

```
print("Data loading functions defined")
```

Data loading functions defined

```
# =====
# CELL 4D – BraTS 2023 GLI data loading
# =====
BRATS23_DIR = '/content/BraTS2023-GLI/ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData'
BRATS23_CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_brats23'
os.makedirs(BRATS23_CACHE_DIR, exist_ok=True)

def build_brats23_dicts(brats_dir):
    folders = sorted([f for f in os.listdir(brats_dir)
                      if f.startswith('BraTS-GLI')])
    print(f"Found {len(folders)} BraTS 2023 GLI folders")
    data_dicts, missing = [], 0
    for folder in folders:
        fp = os.path.join(brats_dir, folder)
        paths = {
            't1': os.path.join(fp, f'{folder}-t1n.nii.gz'),
            't1c': os.path.join(fp, f'{folder}-t1c.nii.gz'),
            't2': os.path.join(fp, f'{folder}-t2w.nii.gz'),
            'flair': os.path.join(fp, f'{folder}-t2f.nii.gz'),
            'label': os.path.join(fp, f'{folder}-seg.nii.gz'),
        }
        if not all(os.path.exists(v) for v in paths.values()):
            missing += 1
            continue
        data_dicts.append({
            'patient_id': folder,
            **paths,
            # No molecular labels in BraTS 2023 GLI
            'idh': float('nan'),
            'grade': float('nan'),
            'mgmt': float('nan'),
        })
    print(f"Loaded: {len(data_dicts)} Missing files: {missing}")
    return data_dicts

def get_brats23_transforms():
    """
```

Same as UCSF transforms but MapLabelValued uses [0,1,2,3]→[0,1,2,3]
 since BraTS 2023 already uses 3 for ET instead of 4.

```
"""
```

```
return Compose([
    LoadImaged(keys=['t1','t1c','t2','flair','label']),
    EnsureChannelFirstd(keys=['t1','t1c','t2','flair','label']),
    ConcatItemsd(keys=['t1','t1c','t2','flair'], name='image'),
    Orientationd(keys=['image','label'], axcodes='RAS'),
    Spacingd(keys=['image','label'], pixdim=cfg.VOXEL_SPACING,
              mode=('bilinear','nearest')),
    NormalizeIntensityd(keys='image', nonzero=True, channel_wise=True),
    CropForegroundd(keys=['image','label'], source_key='image'),
    ResizeWithPadOrCropd(keys=['image','label'],
                          spatial_size=cfg.SPATIAL_SIZE),
    MapLabelValued(keys=['label'],
                   orig_labels=[0,1,2,3], # BraTS 2023: no label 4
                   target_labels=[0,1,2,3]), # already in correct format
    ToTensord(keys=['image','label'])
])
```

```
print("BraTS 2023 GLI functions defined")
print(f"BRATS23_DIR: {BRATS23_DIR}")
print(f"BRATS23_CACHE_DIR: {BRATS23_CACHE_DIR}")
```

```
BraTS 2023 GLI functions defined
BRATS23_DIR: /content/BraTS2023-GLI/ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData
BRATS23_CACHE_DIR: /content/drive/MyDrive/checkpoints/feature_cache_3d_brats23
```

```
# =====
# CELL 5 – SwinUNETR with bottleneck feature extraction
# =====
```

```
def build_swinunetr(weights_path=None):
    model = SwinUNETR(
        in_channels=cfg.IN_CHANNELS,
        out_channels=cfg.OUT_CHANNELS,
        feature_size=cfg.FEATURE_SIZE,
        use_checkpoint=True,
    ).to(cfg.DEVICE)

    if weights_path and os.path.exists(weights_path):
```

```

print(f"Loading: {weights_path}")
ckpt = torch.load(weights_path, map_location=cfg.DEVICE,
                  weights_only=False)
state = ckpt['state_dict']

# Strip common prefixes that cause mismatch
cleaned = {}
for k, v in state.items():
    new_k = k
    for prefix in ['module.', 'model.', 'backbone.']:
        if new_k.startswith(prefix):
            new_k = new_k[len(prefix):]
    cleaned[new_k] = v

model_dict = model.state_dict()

# Separate matched from unmatched for diagnostics
matched = {k:v for k,v in cleaned.items()
           if k in model_dict and model_dict[k].shape==v.shape}
unmatched = {k:v for k,v in cleaned.items()
             if k not in model_dict}
wrong_shape = {k:v for k,v in cleaned.items()
               if k in model_dict and model_dict[k].shape!=v.shape}

model_dict.update(matched)
model.load_state_dict(model_dict)

print(f"Epoch: {ckpt['epoch']} Dice: {ckpt['best_acc']:.4f}")
print(f"Matched:      {len(matched)}/{len(model_dict)}")
print(f"Unmatched:    {len(unmatched)}")
print(f"Wrong shape: {len(wrong_shape)}")

if len(matched) < len(model_dict) * 0.9:
    print("WARNING: fewer than 90% of layers loaded")
    print("Sample unmatched keys:")
    for k in list(unmatched.keys())[:5]:
        print(f"  checkpoint: {k}")
    print("Sample model keys:")
    for k in list(model_dict.keys())[:5]:
        print(f"  model:      {k}")

if cfg.FREEZE_BACKBONE:

```

```

        for p in model.parameters():
            p.requires_grad = False
        model.eval()
        print("Backbone FROZEN")

    print(f"Parameters: {sum(p.numel() for p in model.parameters())/1e6:.1f}M")
    return model

def extract_encoder_features(model, image_tensor):
    """
    Extracts Stage 4 bottleneck features from SwinUNETR.

    SwinViT for 128^3 input returns 5 hidden states:
        [0] [B, 48, 64, 64, 64]
        [1] [B, 96, 32, 32, 32]
        [2] [B, 192, 16, 16, 16]
        [3] [B, 384, 8, 8, 8]
        [4] [B, 768, 4, 4, 4] ← bottleneck, what we use

    Returns: [768] global-average-pooled feature vector
    """
    with torch.no_grad():
        x = image_tensor.unsqueeze(0).to(cfg.DEVICE) # [1,4,128,128,128]
        hidden_states = model.swinViT(x)             # list of 5 tensors
        bottleneck     = hidden_states[4]             # [1,768,4,4,4]
        features       = bottleneck.mean(dim=[2,3,4]) # [1,768]
    return features.squeeze(0).cpu()                 # [768]

swinunetr = build_swinunetr(cfg.WEIGHTS)

```

```

Loading: /content/drive/MyDrive/checkpoints/swinunetr_brats21_fold1.pt
Epoch: 279 Dice: 0.9059
Matched: 159/159
Unmatched: 0
Wrong shape: 0
Backbone FROZEN
Parameters: 62.2M

```

```

# MANDATORY LOADER
ckpt = torch.load(os.path.join(cfg.CKPT_DIR, 'swinunetr_ucsf_finetuned.pt'),

```



```

        map_location=cfg.DEVICE, weights_only=False)
swinunetr.load_state_dict(ckpt['state_dict'])
swinunetr.eval()
print(f"Loaded epoch {ckpt['epoch']} - Val Dice {ckpt['val_dice']:.4f}")

checksum = sum(p.sum().item() for p in swinunetr.parameters())
print(f"Checksum: {checksum:.2f}")
assert abs(checksum - (-34969.76)) < 1.0, "WRONG WEIGHTS LOADED - stop and investigate"

```

```

Loaded epoch 97 - Val Dice 0.8846
Checksum: -34969.76

```

```

# CELL 6
def tta_predict(model, image_tensor, n_runs=cfg.TTA_RUNS):
    """Returns mean_pred [3,H,W,D], conf_map [1,H,W,D], uncertainty."""
    model.eval()
    preds = []
    with torch.no_grad():
        for _ in range(n_runs):
            aug = image_tensor.clone()
            flipped_axes = []
            for axis in [1,2,3]:
                if torch.rand(1).item() > 0.5:
                    aug = torch.flip(aug, dims=[axis])
                    flipped_axes.append(axis)
            aug = aug + (torch.rand(1).item()-0.5)*0.2
            logits = model(aug.unsqueeze(0).to(cfg.DEVICE))
            probs = torch.sigmoid(logits).squeeze(0).cpu()
            # Unflip on the SAME axis index - input and output share
            # the same spatial axis positions (1,2,3)
            for axis in reversed(flipped_axes):
                probs = torch.flip(probs, dims=[axis])
            preds.append(probs)
    stack = torch.stack(preds, dim=0)
    mean_pred= stack.mean(dim=0)
    variance = stack.var(dim=0).mean(0, keepdim=True)
    vmin,vmax= variance.min(), variance.max()
    norm_var = (variance-vmin)/(vmax-vmin+1e-8)
    return mean_pred, 1.0-norm_var, norm_var
print("TTA function corrected and verified - same axis index for flip/unflip")

```

```

TTA function corrected and verified - same axis index for flip/unflip

```

```

# =====
# CELL 7 – Morphological features
# =====

def extract_morphological_features(mean_pred):
    et = (mean_pred[0]>0.3).numpy().astype(np.float32)
    tc = (mean_pred[1]>0.3).numpy().astype(np.float32)
    wt = (mean_pred[2]>0.3).numpy().astype(np.float32)
    edema = np.clip(wt-tc, 0, 1)

    tumor_vol, core_vol = float(wt.sum()), float(tc.sum())
    edema_vol, enh_vol = float(edema.sum()), float(et.sum())
    edema_core_ratio = edema_vol/(core_vol+1e-6)

    eroded = ndimage.binary_erosion(wt.astype(bool))
    surface = wt.astype(bool) & ~eroded
    sa = float(surface.sum())
    compactness = tumor_vol/(sa**1.5+1e-6) if sa>0 else 0.0
    sphericity = ((np.pi**((1/3)*(6*tumor_vol)**(2/3)))/(sa+1e-6)
                  if (sa>0 and tumor_vol>0) else 0.0)

    return {
        'tumor_volume': tumor_vol, 'core_volume': core_vol,
        'edema_volume': edema_vol, 'enhancing_volume': enh_vol,
        'edema_core_ratio': edema_core_ratio, 'surface_area': sa,
        'compactness': compactness, 'sphericity': sphericity,
    }

def morphology_to_tensor(d):
    t = torch.tensor(list(d.values()), dtype=torch.float32)

    # Replace any inf or nan with 0 first
    t = torch.nan_to_num(t, nan=0.0, posinf=0.0, neginf=0.0)

    # log1p the 4 volume features (indices 0-3)
    t[:4] = torch.log1p(t[:4])

    # log1p surface area (index 5)
    t[5] = torch.log1p(t[5])

    # Clamp edema_core_ratio (index 4) – can explode when core_vol ~ 0

```

```

t[4] = torch.clamp(t[4], 0, 10)

# Clamp compactness (index 6) – worst offender, can be billions
t[6] = torch.clamp(t[6], 0, 10)

# Clamp sphericity (index 7) – theoretically 0–1 but can exceed due to voxel discretization
t[7] = torch.clamp(t[7], 0, 2)

return t

print("Morphology functions defined")

```

Morphology functions defined

```

# =====
# CELL 8 – Confidence-Gated Fusion (4 ablation modes)
# =====

class ConfidenceGatedFusion(nn.Module):
    """
    Gate modes:
    'A' encoder only – no fusion
    'B' fixed gate G=0.5
    'C' G = confidence
    'D' G = confidence × sigmoid(W_m · morph)    [proposed]
    """
    def __init__(self, encoder_dim, morph_dim, fused_dim=cfg.FUSED_DIM,
                  mode='D'):
        super().__init__()
        self.mode = mode

        self.encoder_proj = nn.Sequential(
            nn.Linear(encoder_dim, fused_dim),
            nn.ReLU(), nn.Dropout(0.3),
        )
        self.morph_proj = nn.Sequential(
            nn.Linear(morph_dim, 256),
            nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, fused_dim),
        )
        # Morphology-driven gate component (used in mode D)

```

```

        self.morph_gate = nn.Linear(morph_dim, fused_dim)

    def forward(self, encoder_features, morph_features, confidence_score):
        enc = self.encoder_proj(encoder_features)
        morph = self.morph_proj(morph_features)
        conf = confidence_score.view(-1, 1)  # [B,1]

        if self.mode == 'A':
            # encoder only
            return enc

        if self.mode == 'B':
            # fixed equal gate
            G = 0.5
            return G * enc + (1.0 - G) * morph

        if self.mode == 'C':
            # gate = confidence only
            G = conf
            return G * enc + (1.0 - G) * morph

        # mode 'D' - proposed
        # G = confidence × sigmoid(W_m · morph)
        morph_gate = torch.sigmoid(self.morph_gate(morph_features))  # [B,fused]
        G = conf * morph_gate  # [B,fused]
        return G * enc + (1.0 - G) * morph

print("ConfidenceGatedFusion defined")

```

ConfidenceGatedFusion defined

```

# =====
# CELL 9 – Classifier (encoder features + morph + confidence)
# =====

class BrainTumorClassifier(nn.Module):
    def __init__(self, encoder_dim=cfg.ENCODER_DIM,
                  morph_dim=cfg.MORPH_FEATURES,
                  fused_dim=cfg.FUSED_DIM, dropout=0.4,
                  gate_mode=cfg.GATE_MODE):
        super().__init__()

```

```

self.fusion = ConfidenceGatedFusion(
    encoder_dim, morph_dim, fused_dim, mode=gate_mode)
self.shared = nn.Sequential(
    nn.Linear(fused_dim, 256), nn.ReLU(), nn.Dropout(dropout),
)
self.head_idh = nn.Linear(256, 1)
self.head_grade = nn.Linear(256, 1)
self.head_mgmt = nn.Linear(256, 1)

def forward(self, encoder_features, morph_features, confidence_score):
    fused = self.fusion(encoder_features, morph_features,
                        confidence_score)
    shared = self.shared(fused)
    return {
        'idh': self.head_idh(shared),
        'grade': self.head_grade(shared),
        'mgmt': self.head_mgmt(shared),
    }

classifier = BrainTumorClassifier().to(cfg.DEVICE)
print(f"Classifier params: "
      f"{sum(p.numel() for p in classifier.parameters())/1e6:.2f}M")
print(f"Gate mode: {cfg.GATE_MODE}")

```

```

Classifier params: 0.66M
Gate mode: D

```

```

print(f"Pre-cache checksum: {sum(p.sum().item() for p in swinunetr.parameters()):.2f}")
print(f"Expected (fine-tuned epoch 97): should differ from BraTS-only build")

```

```

Pre-cache checksum: -34969.76
Expected (fine-tuned epoch 97): should differ from BraTS-only build

```

```

print(f"Current loaded checksum: {sum(p.sum().item() for p in swinunetr.parameters()):.2f}")
print("Fine-tuned reference:    -34969.76")
print("BraTS-only reference:    -64563.11")

```

```

Current loaded checksum: -34969.76
Fine-tuned reference:    -34969.76
BraTS-only reference:    -64563.11

```

```

# =====
# CELL 9A – Fine-tune SwinUNETR on UCSF-PDGM
# Resuming from epoch 34 (Val Dice 0.8653) with adjusted LR
# =====

import gc
from monai.losses import DiceLoss
from monai.metrics import DiceMetric
from monai.utils.enums import MetricReduction

FINETUNE_EPOCHS    = 100
FINETUNE_LR        = 1e-4      # bumped from 5e-5 to escape plateau
ACCUMULATION_STEPS = 4
FINETUNE_CKPT      = os.path.join(cfg.CKPT_DIR, 'swinunetr_ucsf_finetuned.pt')
LATEST_CKPT        = os.path.join(cfg.CKPT_DIR, 'swinunetr_ucsf_latest.pt')

# — Step 1: Freeze early stages, unfreeze late stages —————
def set_finetune_layers(model):
    for p in model.parameters():
        p.requires_grad = False
    for p in model.swinViT.layers3.parameters():
        p.requires_grad = True
    for p in model.swinViT.layers4.parameters():
        p.requires_grad = True
    for name, p in model.named_parameters():
        if any(k in name for k in [
            'encoder10', 'encoder1', 'encoder2',
            'encoder3', 'encoder4',
            'decoder5', 'decoder4', 'decoder3',
            'decoder2', 'decoder1', 'out'
        ]):
            p.requires_grad = True
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    frozen    = sum(p.numel() for p in model.parameters() if not p.requires_grad)
    print(f"Trainable params: {trainable/1e6:.1f}M")
    print(f"Frozen params:    {frozen/1e6:.1f}M")

set_finetune_layers(swinunetr)
swinunetr.train()

# — Step 2: Define optimizer and scheduler FIRST —————
optimizer = torch.optim.AdamW(

```

```

filter(lambda p: p.requires_grad, swinunetr.parameters()),
lr=FINETUNE_LR, weight_decay=1e-5)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100)

# — Step 3: Force restart from best checkpoint (epoch 34) ———
# Deliberately NOT resuming from LATEST_CKPT (the plateaued
# epoch 47-50 state) – starting fresh from the best known point
# with new LR and optimizer/scheduler state, since the old
# optimizer state was tuned for the now-abandoned LR=5e-5 run.

RESUME      = True
start_epoch = 0
best_val_dice = 0.0
no_improve   = 0
history      = {'train_loss': [], 'val_dice': []}

if RESUME and os.path.exists(FINETUNE_CKPT):
    ckpt = torch.load(FINETUNE_CKPT, map_location=config.DEVICE, weights_only=False)
    swinunetr.load_state_dict(ckpt['state_dict'])
    # Intentionally NOT loading optimizer/scheduler state –
    # fresh optimizer for the new LR regime
    best_val_dice = ckpt['val_dice']
    history       = ckpt.get('history', {'train_loss': [], 'val_dice': []})
    # Continue epoch numbering from where the best model was,
    # not from the plateaued latest checkpoint
    start_epoch   = ckpt['epoch'] + 1
    print(f"Loaded best checkpoint – epoch {ckpt['epoch']}, "
          f"Val Dice {best_val_dice:.4f}")
    print(f"Restarting training from epoch {start_epoch} "
          f"with fresh optimizer at LR={FINETUNE_LR}")
else:
    print("Starting from scratch")

# — Step 4: DataLoaders ———
class SegDataset(torch.utils.data.Dataset):
    def __init__(self, data_dicts, transforms):
        self.data      = data_dicts
        self.transforms = transforms
    def __len__(self):
        return len(self.data)

```

```

def __getitem__(self, idx):
    item = self.data[idx]
    out = self.transforms({
        't1': item['t1'],
        't1c': item['t1c'],
        't2': item['t2'],
        'flair': item['flair'],
        'label': item['label'],
    })
    return out['image'], out['label']

all_dicts = build_ucsf_dicts(cfg.UCSF_CSV, cfg.UCSF_DIR)
train_d, val_d, _ = stratified_split(all_dicts)

train_seg_ds = SegDataset(train_d, get_ucsf_transforms('train'))
val_seg_ds = SegDataset(val_d, get_ucsf_transforms('val'))

train_seg_loader = torch.utils.data.DataLoader(
    train_seg_ds, batch_size=1, shuffle=True,
    num_workers=4, pin_memory=True, persistent_workers=True)
val_seg_loader = torch.utils.data.DataLoader(
    val_seg_ds, batch_size=1, shuffle=False,
    num_workers=4, persistent_workers=True)

print(f"Seg train: {len(train_d)} patients")
print(f"Seg val: {len(val_d)} patients")

# — Step 5: Loss and metric —————
def brats_label_to_channels(label):
    et = (label == 3).float()
    tc = ((label == 1) | (label == 3)).float()
    wt = (label >= 1).float()
    return torch.cat([et, tc, wt], dim=1)

dice_loss = DiceLoss(sigmoid=True, squared_pred=True, reduction='mean')
dice_metric = DiceMetric(
    include_background=False,
    reduction=MetricReduction.MEAN,
    get_not_nans=True)

patience = 25 # more patience for sustained better-GPU run

```



```

print(f"\nStarting fine-tuning from epoch {start_epoch} to {FINETUNE_EPOCHS}...")
print(f"Best val Dice so far: {best_val_dice:.4f}")
print(f"Learning rate: {FINETUNE_LR}")
print(f"Patience: {patience}")
print("="*55)

```

```

# — Step 6: Training loop —————
for epoch in range(start_epoch, FINETUNE_EPOCHS):
    swinunetr.train()
    epoch_loss = 0.0
    optimizer.zero_grad()

    for i, (image, label) in enumerate(tqdm(
        train_seg_loader,
        desc=f"Ep {epoch+1}/{FINETUNE_EPOCHS} [train]",
        leave=False)):
        image = image.to(cfg.DEVICE)
        label = label.to(cfg.DEVICE)
        targets = brats_label_to_channels(label)
        logits = swinunetr(image)
        loss = dice_loss(logits, targets) / ACCUMULATION_STEPS
        loss.backward()
        if ((i+1) % ACCUMULATION_STEPS == 0) or \
            ((i+1) == len(train_seg_loader)):
            optimizer.step()
            optimizer.zero_grad()
        epoch_loss += loss.item() * ACCUMULATION_STEPS

    scheduler.step()
    avg_train_loss = epoch_loss / len(train_seg_loader)

```

```

# — Validation —————
swinunetr.eval()
dice_metric.reset()
with torch.no_grad():
    for image, label in tqdm(
        val_seg_loader,
        desc=f"Ep {epoch+1}/{FINETUNE_EPOCHS} [val]",
        leave=False):
        image = image.to(cfg.DEVICE)
        label = label.to(cfg.DEVICE)
        targets = brats_label_to_channels(label)

```

```

        logits = swinunetr(image)
        preds = (torch.sigmoid(logits) > cfg.SEG_THRESHOLD).float()
        dice_metric(y_pred=preds, y=targets)

val_dice, _ = dice_metric.aggregate()
mean_val_dice = val_dice.mean().item()

history['train_loss'].append(avg_train_loss)
history['val_dice'].append(mean_val_dice)

print(f"Epoch {epoch+1:3d} | "
      f"Train loss: {avg_train_loss:.4f} | "
      f"Val Dice: {mean_val_dice:.4f}")

# — Save best if improved —————
if mean_val_dice > best_val_dice:
    best_val_dice = mean_val_dice
    no_improve = 0
    torch.save({
        'epoch': epoch,
        'state_dict': swinunetr.state_dict(),
        'optimizer': optimizer.state_dict(),
        'scheduler': scheduler.state_dict(),
        'val_dice': best_val_dice,
        'best_val_dice': best_val_dice,
        'no_improve': no_improve,
        'history': history,
    }, FINETUNE_CKPT)
    print(f" ✓ Best model saved (val Dice: {best_val_dice:.4f})")
else:
    no_improve += 1
    print(f" No improve: {no_improve}/{patience}")
    if no_improve >= patience:
        print(f"\nEarly stop at epoch {epoch+1}")
        break

# — Save latest every epoch —————
torch.save({
    'epoch': epoch,
    'state_dict': swinunetr.state_dict(),
    'optimizer': optimizer.state_dict(),
    'scheduler': scheduler.state_dict(),

```

```

        'val_dice':      mean_val_dice,
        'best_val_dice': best_val_dice,
        'no_improve':    no_improve,
        'history':       history,
    }, LATEST_CKPT)

    torch.cuda.empty_cache()
    gc.collect()

# — Step 7: Reload best and verify —————
print("\nReloading best fine-tuned weights...")
ckpt = torch.load(FINETUNE_CKPT, map_location=cfg.DEVICE, weights_only=False)
swinunetr.load_state_dict(ckpt['state_dict'])
swinunetr.eval()
print(f"Loaded epoch {ckpt['epoch']} - Val Dice {ckpt['val_dice']:.4f}")

def compute_dice(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    u = pred.sum() + true.sum()
    return 1.0 if u == 0 else 2.0*(pred & true).sum() / u

sample = train_d[0]
trans = get_ucsf_transforms('val')
out = trans({
    't1': sample['t1'],
    't1c': sample['t1c'],
    't2': sample['t2'],
    'flair': sample['flair'],
    'label': sample['label'],
})
image = out['image']
label = out['label'].squeeze().numpy()

with torch.no_grad():
    logits = swinunetr(image.unsqueeze(0).to(cfg.DEVICE))
    preds = torch.sigmoid(logits).squeeze(0).cpu().numpy()

wt_pred = preds[2] > cfg.SEG_THRESHOLD
wt_true = label >= 1
print(f"\nSanity check on {sample['patient_id']}:")
print(f"  WT Dice:      {compute_dice(wt_pred, wt_true):.4f}")
print(f"  WT pred coverage: {wt_pred.mean()*100:.2f}%")

```

```
print(f" WT true coverage: {wt_true.mean()*100:.2f}%")
print(f"\nLatest checkpoint: {LATEST_CKPT}")
print(f"Best checkpoint: {FINETUNE_CKPT}")
print(f"✓ Ready for next session or cache rebuild")
```

```
Trainable params: 61.7M
Frozen params: 0.5M
/usr/local/lib/python3.12/dist-packages/monai/transforms/spatial/dictionary.py:321: FutureWarning: monai.transforms.spatial.dictionary.warn_deprecated(argname, msg, warning_category)
Loaded best checkpoint – epoch 34, Val Dice 0.8653
Restarting training from epoch 35 with fresh optimizer at LR=0.0001
Patients loaded: 487
Patients skipped: 14
```

```
IDH: 391 wt / 96 mut
Grade: 92 low / 395 high
MGMT: 112 neg / 297 pos
Seg train: 343 patients
Seg val: 72 patients
```

```
Starting fine-tuning from epoch 35 to 100...
Best val Dice so far: 0.8653
Learning rate: 0.0001
Patience: 25
```

```
=====
Epoch 36 | Train loss: 0.2120 | Val Dice: 0.8508
No improve: 1/25
Epoch 37 | Train loss: 0.2086 | Val Dice: 0.8634
No improve: 2/25
Epoch 38 | Train loss: 0.2073 | Val Dice: 0.8557
No improve: 3/25
Epoch 39 | Train loss: 0.2040 | Val Dice: 0.8545
No improve: 4/25
Epoch 40 | Train loss: 0.2067 | Val Dice: 0.8494
No improve: 5/25
Epoch 41 | Train loss: 0.2021 | Val Dice: 0.8504
No improve: 6/25
Epoch 42 | Train loss: 0.2034 | Val Dice: 0.8521
No improve: 7/25
Epoch 43 | Train loss: 0.2010 | Val Dice: 0.8679
✓ Best model saved (val Dice: 0.8679)
Epoch 44 | Train loss: 0.1984 | Val Dice: 0.8553
No improve: 1/25
Epoch 45 | Train loss: 0.1987 | Val Dice: 0.8542
No improve: 2/25
Epoch 46 | Train loss: 0.1967 | Val Dice: 0.8575
```

```

No improve: 3/25
Epoch 47 | Train loss: 0.1973 | Val Dice: 0.8666
No improve: 4/25
Epoch 48 | Train loss: 0.1957 | Val Dice: 0.8726
✓ Best model saved (val Dice: 0.8726)
Epoch 49 | Train loss: 0.1950 | Val Dice: 0.8425
No improve: 1/25
Epoch 50 | Train loss: 0.1945 | Val Dice: 0.8524
No improve: 2/25
Epoch 51 | Train loss: 0.1935 | Val Dice: 0.8488
No improve: 3/25
Epoch 52 | Train loss: 0.1952 | Val Dice: 0.8345
No improve: 4/25
Epoch 53 | Train loss: 0.1919 | Val Dice: 0.8777
✓ Best model saved (val Dice: 0.8777)
Epoch 54 | Train loss: 0.1935 | Val Dice: 0.8740

```

```

print(cfg.UCSF_DIR)
print(cfg.UCSF_CSV)
import os

print(os.path.exists(cfg.UCSF_DIR))
print(os.path.exists(cfg.UCSF_CSV))

```

```

/content/UCSF-PDGM-v5
/content/drive/MyDrive/UCSF-PDGM-metadata_v5.csv
True
True

```

```

# =====
# CELL 10 – Loss functions
# =====

import torch.nn as nn

IDH_W = torch.tensor([2.5]).to(cfg.DEVICE)
GRADE_W = torch.tensor([92/395]).to(cfg.DEVICE)
MGMT_W = torch.tensor([0.395]).to(cfg.DEVICE)

print(f"IDH_W: {IDH_W.item():.4f}")
print(f"GRADE_W: {GRADE_W.item():.4f}")
print(f"MGMT_W: {MGMT_W.item():.4f}")

idh_crit = nn.BCEWithLogitsLoss(pos_weight=IDH_W)

```

```

grade_crit = nn.BCEWithLogitsLoss(pos_weight=GRADE_W)
mgmt_crit = nn.BCEWithLogitsLoss(pos_weight=MGMT_W)

print(f"grade_crit pos_weight: {grade_crit.pos_weight.item():.4f}")

TASK_W = {'idh':1.0, 'grade':1.0, 'mgmt':0.8}

def classification_loss(preds, labels):
    loss = 0.0
    idh_valid = ~torch.isnan(labels['idh'])
    if idh_valid.any():
        loss += TASK_W['idh'] * idh_crit(
            preds['idh'][idh_valid], labels['idh'][idh_valid].float())
    grade_valid = ~torch.isnan(labels['grade'])
    if grade_valid.any():
        loss += TASK_W['grade'] * grade_crit(
            preds['grade'][grade_valid], labels['grade'][grade_valid].float())
    mgmt_valid = ~torch.isnan(labels['mgmt'])
    if mgmt_valid.any():
        loss += TASK_W['mgmt'] * mgmt_crit(
            preds['mgmt'][mgmt_valid], labels['mgmt'][mgmt_valid].float())
    return loss

print("Loss functions defined – all weights confirmed correct")

```

```

IDH_W: 2.5000
GRADE_W: 0.2329
MGMT_W: 0.3950
grade_crit pos_weight: 0.2329
Loss functions defined – all weights confirmed correct

```

```

# CHECK ONCE BEFORE CREATING A NEW CACHE – NOT IMP BUT DO IT ANYWAY
print(sum(p.sum().item() for p in swinunetr.parameters()))

```

```

-34969.759021766484

```

```

# DELETING FILES BEFORE CACHE CONSTRUCTION
import shutil

# Full feature cache (Drive)
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
if os.path.exists(CACHE_DIR):

```

```

    shutil.rmtree(CACHE_DIR)
os.makedirs(CACHE_DIR)

# Slim cache, both copies
SLIM_CACHE_LOCAL = '/content/feature_cache_slim_ucsf'
SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
for d in [SLIM_CACHE_LOCAL, SLIM_CACHE_DRIVE]:
    if os.path.exists(d):
        shutil.rmtree(d)

```

```

# =====
# CELL 11 – Precompute feature cache
# Stores [768] encoder features + morph + confidence per patient
# Tiny: ~3KB per patient, ~1.5MB total for all 487
# =====

import gc
from tqdm import tqdm

def precompute_feature_cache(model, csv_path, ucsf_dir, cache_dir, n_runs=cfg.TTA_RUNS):
    all_dicts = build_ucsf_dicts(csv_path, ucsf_dir)
    transforms = get_ucsf_transforms('val')
    model.eval()
    computed, skipped, failed = 0, 0, 0
    for item in tqdm(all_dicts, desc="Precomputing 3D features"):
        pid = item['patient_id']
        path = os.path.join(cache_dir, f"{pid}.pt")
        if os.path.exists(path):
            skipped += 1
            continue
        try:
            sample = transforms({
                't1':item['t1'], 't1c':item['t1c'],
                't2':item['t2'], 'flair':item['flair'],
                'label':item['label'],
            })
            image = sample['image']
            enc_feat = extract_encoder_features(model, image)
            mean_pred, conf_map, _ = tta_predict(model, image, n_runs)
            conf_score = torch.tensor([conf_map.mean().item()])
            morph = morphology_to_tensor(extract_morphological_features(mean_pred))

```

```

        torch.save({
            'patient_id': pid,
            'encoder': enc_feat,
            'morph': morph,
            'conf_score': conf_score,
            'mean_pred': mean_pred,
            'label': sample['label'],
            'idh': torch.tensor([item['idh']], dtype=torch.float32),
            'grade': torch.tensor([item['grade']], dtype=torch.float32),
            'mgmt': torch.tensor([item['mgmt']], dtype=torch.float32),
        }, path)
        computed += 1
        torch.cuda.empty_cache(); gc.collect()
    except Exception as e:
        print(f"Failed {pid}: {e}")
        failed += 1
        continue
    print(f"\nComputed: {computed} Skipped: {skipped} Failed: {failed}")

```

```
precompute_feature_cache(swinunetr, cfg.UCSF_CSV, cfg.UCSF_DIR, CACHE_DIR)
```

```
/usr/local/lib/python3.12/dist-packages/monai/utils/deprecate_utils.py:321: FutureWarning: monai.transforms.spatial.dictionary
```

```
warn_deprecated(argname, msg, warning_category)
```

```
Patients loaded: 487
```

```
Patients skipped: 14
```

```
IDH: 391 wt / 96 mut
```

```
Grade: 92 low / 395 high
```

```
MGMT: 112 neg / 297 pos
```

```
Precomputing 3D features: 100%|██████████| 487/487 [27:57<00:00, 3.44s/it]
```

```
Computed: 487 Skipped: 0 Failed: 0
```

```

# =====
# CELL 11B – Recompute morphology at threshold 0.3 (robust version)
# =====

```

```
cfg.SEG_THRESHOLD = 0.3
```

```

from tqdm import tqdm
import time

```



```

CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
def list_cache_files():
    for attempt in range(5):
        try:
            return [f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')]
        except OSError:
            print("Listing failed, remounting...")
            drive.mount('/content/drive', force_remount=True)
            time.sleep(3)
    raise RuntimeError("Could not list cache dir")

files = list_cache_files()
print(f"Recomputing morphology for {len(files)} patients "
      f"at threshold {cfg.SEG_THRESHOLD}...")

failed = []
for fname in tqdm(files, desc="Recomputing morphology"):
    path = os.path.join(CACHE_DIR, fname)

    for attempt in range(4):
        try:
            d = torch.load(path, map_location='cpu', weights_only=False)
            d['morph'] = morphology_to_tensor(
                extract_morphological_features(d['mean_pred']))
            torch.save(d, path)
            break # success
        except OSError as e:
            if attempt < 3:
                drive.mount('/content/drive', force_remount=True)
                time.sleep(3)
            else:
                print(f"\nFailed permanently: {fname}: {e}")
                failed.append(fname)

print(f"\nDone. Failed: {len(failed)}")
if failed:
    print("Failed files:", failed[:10])

```

```

Recomputing morphology for 487 patients at threshold 0.3...
Recomputing morphology: 1%|          | 4/487 [00:10<22:00, 2.73s/it]Mounted at /content/drive
Recomputing morphology: 100%|██████████| 487/487 [22:52<00:00, 2.82s/it]
Done. Failed: 0

```

```
import gc
```

```
import shutil, zipfile, time, os

ZIP_DRIVE = '/content/drive/MyDrive/BraTS2023-GLI/ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData.zip'
LOCAL_ZIP = '/content/brats23.zip'
LOCAL_DIR = '/content/BraTS2023-GLI'

print("Copying zip from Drive...")
start = time.time()
shutil.copy2(ZIP_DRIVE, LOCAL_ZIP)
print(f"Copy done in {time.time()-start:.1f}s")

print("Extracting...")
start = time.time()
with zipfile.ZipFile(LOCAL_ZIP, 'r') as zf:
    zf.extractall(LOCAL_DIR)
print(f"Extract done in {time.time()-start:.1f}s")
os.remove(LOCAL_ZIP)

BRATS23_DIR = f'{LOCAL_DIR}/ASNR-MICCAI-BraTS2023-GLI-Challenge-TrainingData'
print(f"Ready: {len(os.listdir(BRATS23_DIR))} folders")
```

```
Copying zip from Drive...
Copy done in 473.8s
Extracting...
Extract done in 176.4s
Ready: 1251 folders
```

```
# =====
# CELL 11D – Precompute BraTS 2023 feature cache
# Segmentation only – no molecular labels
# =====

# Verify fine-tuned weights loaded
checksum = sum(p.sum().item() for p in swinunetr.parameters())
print(f"Pre-BraTS23-cache checksum: {checksum:.2f}")
assert abs(checksum - (-34969.76)) < 1.0, "WRONG WEIGHTS – stop"

def precompute_brats23_cache(model, brats_dir, cache_dir,
```

```

        n_runs=cfg.TTA_RUNS):
all_dicts = build_brats23_dicts(brats_dir)
transforms = get_brats23_transforms()
model.eval()
computed, skipped, failed = 0, 0, 0

for item in tqdm(all_dicts, desc="Precomputing BraTS23 features"):
    pid = item['patient_id']
    path = os.path.join(cache_dir, f"{pid}.pt")
    if os.path.exists(path):
        skipped += 1
        continue
    try:
        sample = transforms({
            't1': item['t1'], 't1c': item['t1c'],
            't2': item['t2'], 'flair': item['flair'],
            'label': item['label'],
        })
        image = sample['image']
        enc_feat = extract_encoder_features(model, image)
        mean_pred, conf_map, _ = tta_predict(model, image, n_runs)
        conf_score = torch.tensor([conf_map.mean().item()])
        morph = morphology_to_tensor(
            extract_morphological_features(mean_pred))
        torch.save({
            'patient_id': pid,
            'encoder': enc_feat,
            'morph': morph,
            'conf_score': conf_score,
            'mean_pred': mean_pred,
            'label': sample['label'],
        }, path)
        computed += 1
        torch.cuda.empty_cache()
        gc.collect()
    except Exception as e:
        print(f"Failed {pid}: {e}")
        failed += 1

n = len([f for f in os.listdir(cache_dir) if f.endswith('.pt')])
print(f"\nComputed: {computed} Skipped: {skipped} Failed: {failed}")
print(f"Total BraTS23 cache files: {n}")

```

```
precompute_brats23_cache(swinunetr, BRATS23_DIR, BRATS23_CACHE_DIR)
```

```
Pre-BraTS23-cache checksum: -34969.76
Found 1251 BraTS 2023 GLI folders
Loaded: 1251 Missing files: 0
Precomputing BraTS23 features: 100%|██████████| 1251/1251 [1:08:55<00:00, 3.31s/it]
Computed: 1189 Skipped: 62 Failed: 0
Total BraTS23 cache files: 1251
```

```
FINETUNE_CKPT = os.path.join(cfg.CKPT_DIR, 'swinunetr_ucsf_finetuned.pt')
```

```
ckpt = torch.load(FINETUNE_CKPT, map_location='cpu', weights_only=False)
print(f"Current best checkpoint: epoch {ckpt['epoch']}, Val Dice {ckpt['val_dice']:.4f}")
```

```
Current best checkpoint: epoch 97, Val Dice 0.8846
```

```
# CHECK BEFORE SLIM CACHE -----
all_dicts = build_ucsf_dicts(cfg.UCSF_CSV, cfg.UCSF_DIR)
cached = {f.replace('.pt', '') for f in os.listdir(CACHE_DIR) if f.endswith('.pt')}
all_dicts = [d for d in all_dicts if d['patient_id'] in cached]
_, _, test_d = stratified_split(all_dicts)

dices = []
for item in test_d:
    d = torch.load(os.path.join(CACHE_DIR, f"{item['patient_id']}.pt"),
                  map_location='cpu', weights_only=False)
    mp, lab = d['mean_pred'].numpy(), d['label'].squeeze().numpy()
    wt_pred, wt_true = mp[2] > 0.3, lab >= 1
    u = wt_pred.sum() + wt_true.sum()
    dices.append(2*(wt_pred & wt_true).sum()/u if u > 0 else 1.0)
dices = np.array(dices)
print(f"Mean: {dices.mean():.4f} Median: {np.median(dices):.4f}")
```

```
Patients loaded: 487
Patients skipped: 14
```

```
IDH: 391 wt / 96 mut
Grade: 92 low / 395 high
MGMT: 112 neg / 297 pos
Mean: 0.8655 Median: 0.9204
```

```
# CELL 12

import shutil, os
from tqdm import tqdm
import torch

# Delete old slim cache
SLIM_CACHE = '/content/feature_cache_slim_ucsf'
if os.path.exists(SLIM_CACHE):
    shutil.rmtree(SLIM_CACHE)
os.makedirs(SLIM_CACHE)
print("Old slim cache deleted")

# Rebuild from new full cache
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
files = [f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')]
print(f"Rebuilding from {len(files)} full cache files...")

failed = []
for fname in tqdm(files, desc="Building slim cache"):
    try:
        d = torch.load(os.path.join(CACHE_DIR, fname),
                        map_location='cpu', weights_only=False)
        torch.save({
            'patient_id': d['patient_id'],
            'encoder': d['encoder'],
            'morph': d['morph'],
            'conf_score': d['conf_score'],
            'idh': d['idh'],
            'grade': d['grade'],
            'mgmt': d['mgmt'],
        }, os.path.join(SLIM_CACHE, fname))
    except Exception as e:
        print(f"Failed {fname}: {e}")
        failed.append(fname)

print(f"Done. Failed: {len(failed)}")

# Verify
import numpy as np
max_vals = []
for fname in os.listdir(SLIM_CACHE):
```

```

    if not fname.endswith('.pt'):
        continue
    d = torch.load(os.path.join(SLIM_CACHE, fname),
                      map_location='cpu', weights_only=False)
    max_vals.append(d['morph'].max().item())

max_vals = np.array(max_vals)
print(f"\nAfter rebuild:")
print(f"Morph max - mean: {max_vals.mean():.3f}  max: {max_vals.max():.3f}")
print(f"Files above 15: {(max_vals > 15).sum()}")
if (max_vals > 15).sum() == 0:
    print("All clean - safe to train")
else:
    print("Still dirty - check full cache")

# Copy fresh slim cache to Drive as the new backup
import time
SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
if os.path.exists(SLIM_CACHE_DRIVE):
    shutil.rmtree(SLIM_CACHE_DRIVE)
    print("Removed stale Drive backup")

print("Copying fresh slim cache to Drive...")
start = time.time()
shutil.copytree(SLIM_CACHE, SLIM_CACHE_DRIVE)
print(f"Done in {time.time()-start:.1f}s - {len(os.listdir(SLIM_CACHE_DRIVE))} files on Drive")

```

```

Old slim cache deleted
Rebuilding from 487 full cache files...
Building slim cache: 100%|██████████| 487/487 [09:37<00:00, 1.19s/it]
Done. Failed: 0

```

```

After rebuild:
Morph max - mean: 11.148  max: 12.594
Files above 15: 0
All clean - safe to train
Copying fresh slim cache to Drive...
Done in 4.4s - 487 files on Drive

```

```

import shutil, time, os

SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
SLIM_CACHE_LOCAL = '/content/feature_cache_slim_ucsf'

```

```

if not os.path.exists(SLIM_CACHE_LOCAL):
    print("Copying slim cache from Drive...")
    start = time.time()
    shutil.copytree(SLIM_CACHE_DRIVE, SLIM_CACHE_LOCAL)
    print(f"Done in {time.time()-start:.1f}s")
else:
    print(f"Already local - {len(os.listdir(SLIM_CACHE_LOCAL))} files")

SLIM_CACHE = SLIM_CACHE_LOCAL

```

```

# Copy fresh slim cache to Drive as the new backup
import time
SLIM_CACHE_DRIVE = '/content/drive/MyDrive/checkpoints/feature_cache_slim_ucsf'
if os.path.exists(SLIM_CACHE_DRIVE):
    shutil.rmtree(SLIM_CACHE_DRIVE)
    print("Removed stale Drive backup")

print("Copying fresh slim cache to Drive...")
start = time.time()
shutil.copytree(SLIM_CACHE, SLIM_CACHE_DRIVE)
print(f"Done in {time.time()-start:.1f}s - {len(os.listdir(SLIM_CACHE_DRIVE))} files on Drive")

```

```

Removed old Drive slim cache
Copying clean slim cache to Drive...
Done in 3.7 seconds
Files on Drive: 487

```

```

# =====
# CELL 13 - Classifier training with early stopping
# =====

class CachedDataset(torch.utils.data.Dataset):
    def __init__(self, files):
        self.files = files
    def __len__(self):
        return len(self.files)
    def __getitem__(self, idx):
        d = torch.load(self.files[idx], map_location='cpu',
                        weights_only=False)
        return {

```

```

        'encoder': d['encoder'],
        'morph': d['morph'],
        'conf_score': d['conf_score'].view(1),
        'idh': d['idh'], 'grade': d['grade'], 'mgmt': d['mgmt'],
        'patient_id': d['patient_id'],
    }

```

```

def build_loaders(cache_dir, csv_path, ucsf_dir):
    all_dicts = build_ucsf_dicts(csv_path, ucsf_dir)
    cached_pids = {f.replace('.pt', '') for f in os.listdir(cache_dir)
                    if f.endswith('.pt')}
    all_dicts = [d for d in all_dicts if d['patient_id'] in cached_pids]
    print(f"Patients with cache: {len(all_dicts)}")

    train_d, val_d, test_d = stratified_split(all_dicts)
    def paths(dicts):
        return [os.path.join(cache_dir, f"{d['patient_id']}.pt")
                for d in dicts]

    print(f"Split: {len(train_d)} train / {len(val_d)} val / {len(test_d)} test")

    tl = torch.utils.data.DataLoader(
        CachedDataset(paths(train_d)), batch_size=cfg.BATCH_SIZE,
        shuffle=True, num_workers=0, pin_memory=True)
    vl = torch.utils.data.DataLoader(
        CachedDataset(paths(val_d)), batch_size=cfg.BATCH_SIZE,
        shuffle=False, num_workers=0)
    tel = torch.utils.data.DataLoader(
        CachedDataset(paths(test_d)), batch_size=1,
        shuffle=False, num_workers=0)
    return tl, vl, tel

def train_classifier(clf_model, cache_dir, csv_path, ucsf_dir,
                    epochs=cfg.CLF_EPOCHS, patience=20,
                    tag=cfg.GATE_MODE):
    train_loader, val_loader, _ = build_loaders(cache_dir, csv_path, ucsf_dir)

    optimizer = torch.optim.AdamW(clf_model.parameters(),
                                   lr=cfg.LR, weight_decay=1e-5)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(

```



```

optimizer, T_max=epochs)

history = {'train_loss':[], 'val_loss':[], 'train_acc':[], 'val_acc':[]}
best_ckpt = os.path.join(cfg.CKPT_DIR, f'clf_best_{tag}.pth')
latest_ckpt = os.path.join(cfg.CKPT_DIR, f'clf_latest_{tag}.pth')

best_val, no_improve = float('inf'), 0

# fresh start
for ck in [best_ckpt, latest_ckpt]:
    if os.path.exists(ck):
        os.remove(ck)
print(f"Training variant {tag} from scratch")

for epoch in range(epochs):
    clf_model.train()
    ep_loss, ep_corr, ep_tot = 0.0, 0, 0
    for batch in tqdm(train_loader, desc=f"Ep {epoch+1}/{epochs}",
                      leave=False):
        enc = batch['encoder'].to(cfg.DEVICE)
        mor = batch['morph'].to(cfg.DEVICE)
        conf = batch['conf_score'].to(cfg.DEVICE)
        lbl = {k: batch[k].to(cfg.DEVICE)
               for k in ['idh', 'grade', 'mgmt']}
        optimizer.zero_grad()
        preds = clf_model(enc, mor, conf)
        loss = classification_loss(preds, lbl)
        loss.backward()
        optimizer.step()
        ep_loss += loss.item()
        v = ~torch.isnan(lbl['idh'])
        if v.any():
            p = torch.sigmoid(preds['idh'][v])>0.5
            t = lbl['idh'][v]>0.5
            ep_corr += (p==t).sum().item(); ep_tot += v.sum().item()
    scheduler.step()
    tr_loss = ep_loss/len(train_loader)
    tr_acc = ep_corr/ep_tot if ep_tot>0 else 0.0

    clf_model.eval()
    v_loss, v_corr, v_tot = 0.0, 0, 0
    with torch.no_grad():

```

```

for batch in val_loader:
    enc = batch['encoder'].to(cfg.DEVICE)
    mor = batch['morph'].to(cfg.DEVICE)
    conf = batch['conf_score'].to(cfg.DEVICE)
    lbl = {k: batch[k].to(cfg.DEVICE)
            for k in ['idh', 'grade', 'mgmt']}
    preds = clf_model(enc, mor, conf)
    v_loss += classification_loss(preds, lbl).item()
    vv = ~torch.isnan(lbl['idh'])
    if vv.any():
        p = torch.sigmoid(preds['idh'][vv])>0.5
        t = lbl['idh'][vv]>0.5
        v_corr += (p==t).sum().item(); v_tot += vv.sum().item()
va_loss = v_loss/len(val_loader)
va_acc = v_corr/v_tot if v_tot>0 else 0.0

for k, val in [('train_loss', tr_loss), ('val_loss', va_loss),
               ('train_acc', tr_acc), ('val_acc', va_acc)]:
    history[k].append(val)

print(f"Ep {epoch+1:3d} | Train {tr_loss:.4f} acc {tr_acc:.3f} "
      f"| Val {va_loss:.4f} acc {va_acc:.3f}")

torch.save({'epoch': epoch, 'model_state': clf_model.state_dict(),
           'history': history, 'best_val_loss': best_val}, latest_ckpt)

if va_loss < best_val:
    best_val, no_improve = va_loss, 0
    torch.save({'epoch': epoch, 'model_state': clf_model.state_dict(),
               'val_loss': best_val, 'history': history}, best_ckpt)
    print(f" ✓ Best saved (val {best_val:.4f})")
else:
    no_improve += 1
    if no_improve >= patience:
        print(f"\nEarly stop at epoch {epoch+1}")
        break

print(f"\nDone variant {tag}. Best val: {best_val:.4f}")
return clf_model, history

# Run (gate mode D, the proposed variant):
classifier, history = train_classifier(

```

classifier, SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)

Patients loaded: 487

Patients skipped: 14

IDH: 391 wt / 96 mut

Grade: 92 low / 395 high

MGMT: 112 neg / 297 pos

Patients with cache: 487

Split: 343 train / 72 val / 72 test

Training variant D from scratch

Ep 1 | Train 1.2331 acc 0.784 | Val 0.8846 acc 0.903

✓ Best saved (val 0.8846)

Ep 2 | Train 1.0619 acc 0.854 | Val 0.8143 acc 0.903

✓ Best saved (val 0.8143)

Ep 3 | Train 1.0441 acc 0.845 | Val 0.9386 acc 0.847

Ep 4 | Train 1.0007 acc 0.837 | Val 0.8311 acc 0.917

Ep 5 | Train 1.0193 acc 0.825 | Val 0.8562 acc 0.903

Ep 6 | Train 1.0557 acc 0.831 | Val 0.8391 acc 0.889

Ep 7 | Train 0.9970 acc 0.822 | Val 0.8084 acc 0.903

✓ Best saved (val 0.8084)

Ep 8 | Train 1.0036 acc 0.831 | Val 0.7998 acc 0.903

✓ Best saved (val 0.7998)

Ep 9 | Train 0.9887 acc 0.843 | Val 0.8440 acc 0.903

Ep 10 | Train 1.0241 acc 0.816 | Val 0.7898 acc 0.903

✓ Best saved (val 0.7898)

Ep 11 | Train 0.9593 acc 0.822 | Val 0.8389 acc 0.875

Ep 12 | Train 0.9784 acc 0.834 | Val 0.8544 acc 0.903

Ep 13 | Train 0.9805 acc 0.848 | Val 0.8257 acc 0.903

Ep 14 | Train 1.0104 acc 0.843 | Val 0.9161 acc 0.819

Ep 15 | Train 0.9754 acc 0.851 | Val 0.7804 acc 0.903

✓ Best saved (val 0.7804)

Ep 16 | Train 0.9335 acc 0.860 | Val 0.7560 acc 0.931

✓ Best saved (val 0.7560)

Ep 17 | Train 0.9577 acc 0.845 | Val 0.8357 acc 0.889

Ep 18 | Train 0.9706 acc 0.840 | Val 0.7916 acc 0.917

Ep 19 | Train 0.9666 acc 0.860 | Val 0.7697 acc 0.903

Ep 20 | Train 0.9458 acc 0.860 | Val 0.8008 acc 0.903

Ep 21 | Train 0.9075 acc 0.863 | Val 0.7632 acc 0.917

Ep 22 | Train 0.9247 acc 0.872 | Val 0.8490 acc 0.806

Ep 23 | Train 0.9458 acc 0.845 | Val 0.7955 acc 0.931

Ep 24 | Train 0.9095 acc 0.872 | Val 0.7849 acc 0.917

Ep 25 | Train 0.9218 acc 0.863 | Val 0.7642 acc 0.931

Ep 26 | Train 0.9065 acc 0.883 | Val 0.8183 acc 0.875

Ep 27 | Train 0.9057 acc 0.869 | Val 0.7801 acc 0.917

```

Ep 28 | Train 0.8983 acc 0.883 | Val 0.7762 acc 0.917
Ep 29 | Train 0.9039 acc 0.869 | Val 0.7536 acc 0.917
✓ Best saved (val 0.7536)
Ep 30 | Train 0.8700 acc 0.883 | Val 0.7553 acc 0.931
Ep 31 | Train 0.9229 acc 0.869 | Val 0.7546 acc 0.931
Ep 32 | Train 0.8702 acc 0.866 | Val 0.7658 acc 0.931
Ep 33 | Train 0.8739 acc 0.872 | Val 0.7990 acc 0.875
Ep 34 | Train 0.8815 acc 0.872 | Val 0.7601 acc 0.903
Ep 35 | Train 0.9072 acc 0.892 | Val 0.7680 acc 0.931
Ep 36 | Train 0.8423 acc 0.895 | Val 0.7642 acc 0.889
Ep 37 | Train 0.8935 acc 0.863 | Val 0.7538 acc 0.931
Ep 38 | Train 0.8906 acc 0.878 | Val 0.7609 acc 0.903
Ep 39 | Train 0.8865 acc 0.886 | Val 0.7573 acc 0.931
Ep 40 | Train 0.8680 acc 0.875 | Val 0.7509 acc 0.931
✓ Best saved (val 0.7509)

```

```

# =====
# CELL 14 – Run full ablation study (variants A, B, C, D)
# =====

def run_ablation(cache_dir, csv_path, ucsf_dir):
    """Trains all 4 gate variants and returns their histories."""
    results = {}
    for mode in ['A', 'B', 'C', 'D']:
        print(f"\n{'='*55}\n  ABLATION VARIANT {mode}\n{'='*55}")
        model = BrainTumorClassifier(gate_mode=mode).to(cfg.DEVICE)
        _, hist = train_classifier(model, cache_dir, csv_path, ucsf_dir,
                                   tag=mode)

        results[mode] = hist
    return results

ablation_results = run_ablation(SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)

```

```

=====
ABLATION VARIANT A
=====
Patients loaded: 487
Patients skipped: 14

IDH:   391 wt / 96 mut
Grade: 92 low / 395 high
MGMT:  112 neg / 297 pos
Patients with cache: 487

```

```

Split: 343 train / 72 val / 72 test
Training variant A from scratch
Ep   1 | Train 1.4782 acc 0.796 | Val 1.4088 acc 0.806
  ✓ Best saved (val 1.4088)
Ep   2 | Train 1.4473 acc 0.799 | Val 1.4105 acc 0.806
Ep   3 | Train 1.4664 acc 0.723 | Val 1.4121 acc 0.806
Ep   4 | Train 1.4582 acc 0.802 | Val 1.4069 acc 0.806
  ✓ Best saved (val 1.4069)
Ep   5 | Train 1.4411 acc 0.802 | Val 1.4096 acc 0.806
Ep   6 | Train 1.4475 acc 0.802 | Val 1.4066 acc 0.806
  ✓ Best saved (val 1.4066)
Ep   7 | Train 1.4415 acc 0.802 | Val 1.4070 acc 0.806
Ep   8 | Train 1.4450 acc 0.802 | Val 1.4059 acc 0.806
  ✓ Best saved (val 1.4059)
Ep   9 | Train 1.4402 acc 0.802 | Val 1.4053 acc 0.806
  ✓ Best saved (val 1.4053)
Ep  10 | Train 1.4527 acc 0.802 | Val 1.4067 acc 0.806
Ep  11 | Train 1.4412 acc 0.802 | Val 1.4081 acc 0.806
Ep  12 | Train 1.4519 acc 0.802 | Val 1.4089 acc 0.806
Ep  13 | Train 1.4368 acc 0.802 | Val 1.4068 acc 0.806
Ep  14 | Train 1.4524 acc 0.802 | Val 1.4082 acc 0.806
Ep  15 | Train 1.4527 acc 0.802 | Val 1.4133 acc 0.806
Ep  16 | Train 1.4439 acc 0.802 | Val 1.4081 acc 0.806
Ep  17 | Train 1.4395 acc 0.802 | Val 1.4116 acc 0.806
Ep  18 | Train 1.4306 acc 0.802 | Val 1.4073 acc 0.806
Ep  19 | Train 1.4564 acc 0.802 | Val 1.4090 acc 0.806
Ep  20 | Train 1.4440 acc 0.802 | Val 1.4093 acc 0.806
Ep  21 | Train 1.4419 acc 0.802 | Val 1.4083 acc 0.806
Ep  22 | Train 1.4523 acc 0.802 | Val 1.4107 acc 0.806
Ep  23 | Train 1.4429 acc 0.802 | Val 1.4106 acc 0.806
Ep  24 | Train 1.4378 acc 0.802 | Val 1.4099 acc 0.806
Ep  25 | Train 1.4492 acc 0.802 | Val 1.4085 acc 0.806
Ep  26 | Train 1.4397 acc 0.802 | Val 1.4093 acc 0.806
Ep  27 | Train 1.4446 acc 0.802 | Val 1.4118 acc 0.806
Ep  28 | Train 1.4364 acc 0.802 | Val 1.4082 acc 0.806
Ep  29 | Train 1.4351 acc 0.802 | Val 1.4079 acc 0.806

```

Early stop at epoch 29

Done variant A. Best val: 1.4053

```

=====
ABLATION VARIANT B
=====

```

Patients loaded: 487

Patients skipped: 14

```
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
```

```
print(SLIM_CACHE)
print(len([f for f in os.listdir(SLIM_CACHE) if f.endswith('.pt')]))
```

```
/content/feature_cache_slim_ucsf
487
```

```
# =====
# CELL 15 A – Evaluation (inline display, classification + segmentation)
# =====

import matplotlib.pyplot as plt
from scipy import ndimage

# — Metric functions —————
def compute_dice(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    u = pred.sum()+true.sum()
    return 1.0 if u==0 else 2.0*(pred&true).sum()/u

def compute_hd95(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    if pred.sum()==0 or true.sum()==0:
        return float('nan')
    ps = pred & ~binary_erosion(pred)
    ts = true & ~binary_erosion(true)
    d1 = distance_transform_edt(~true)[ps]
    d2 = distance_transform_edt(~pred)[ts]
    return max(np.percentile(d1,95), np.percentile(d2,95))

def compute_iou(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    intersection = (pred & true).sum()
    union = (pred | true).sum()
    return 1.0 if union==0 else intersection/union

def compute_precision_seg(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    if pred.sum()==0:
        return 1.0 if true.sum()==0 else 0.0
```

```

    return (pred & true).sum() / pred.sum()

def compute_recall_seg(pred, true):
    pred, true = pred.astype(bool), true.astype(bool)
    if true.sum()==0:
        return 1.0 if pred.sum()==0 else 0.0
    return (pred & true).sum() / true.sum()

def keep_largest_component(mask):
    labeled, n = ndimage.label(mask)
    if n <= 1:
        return mask
    sizes = ndimage.sum(mask, labeled, range(1, n+1))
    largest = np.argmax(sizes) + 1
    return labeled == largest

# — Classification evaluator —————
def evaluate_classifier(clf_model, test_loader):
    clf_model.eval()
    res = {'t': {'probs': [], 'labels': []}}
    for t in ['idh', 'grade', 'mgmt']:
        with torch.no_grad():
            for batch in tqdm(test_loader, desc="Classification eval"):
                enc = batch['encoder'].to(cfg.DEVICE)
                mor = batch['morph'].to(cfg.DEVICE)
                conf = batch['conf_score'].to(cfg.DEVICE)
                preds = clf_model(enc, mor, conf)
                for t in ['idh', 'grade', 'mgmt']:
                    res[t]['probs'].extend(torch.sigmoid(preds[t]).squeeze(-1).cpu().tolist())
                    res[t]['labels'].extend(batch[t].squeeze(-1).cpu().tolist())

    metrics = {}
    names = {'idh': 'IDH Status', 'grade': 'WHO Grade', 'mgmt': 'MGMT'}
    for t in ['idh', 'grade', 'mgmt']:
        p = np.array(res[t]['probs'])
        l = np.array(res[t]['labels'])
        valid = ~np.isnan(l)
        p, l = p[valid], l[valid]
        if len(l)==0: continue
        pb = (p>=0.5).astype(int)
        try: auc = roc_auc_score(l, p)
        except: auc = float('nan')

```

```

tn,fp,fn,tp = confusion_matrix(l,pb,labels=[0,1]).ravel()
metrics[t] = {
    'accuracy': accuracy_score(l,pb),
    'precision': precision_score(l,pb,zero_division=0),
    'recall': recall_score(l,pb,zero_division=0),
    'f1': f1_score(l,pb,zero_division=0),
    'auc': auc,
    'sensitivity': tp/(tp+fn) if (tp+fn)>0 else 0,
    'specificity': tn/(tn+fp) if (tn+fp)>0 else 0,
    'tp':tp,'tn':tn,'fp':fp,'fn':fn,
    'probs':p,'labels':l,'preds':pb,
    'name':names[t],'n':len(l)
}
return metrics

```

```

# — Segmentation evaluator —————
def evaluate_segmentation(full_cache_dir, csv_path, ucsf_dir):
    all_dicts = build_ucsf_dicts(csv_path, ucsf_dir)
    cached = {f.replace('.pt','') for f in os.listdir(full_cache_dir)
                if f.endswith('.pt')}
    all_dicts = [d for d in all_dicts if d['patient_id'] in cached]
    _,_,test_d = stratified_split(all_dicts)
    res = {r:{'dice':[],'hd95':[],'iou':[],'prec':[],'rec':[]}}
    for r in ['ET','TC','WT']:
        for item in tqdm(test_d, desc="Segmentation eval"):
            try:
                d = torch.load(os.path.join(full_cache_dir,
                                              f"{item['patient_id']}.pt"),
                               map_location='cpu', weights_only=False)
            except: continue
            if 'mean_pred' not in d: continue
            mp = d['mean_pred'].numpy()
            lab = d['label'].squeeze().numpy()
            for name,pred,true in [
                ('ET', mp[0]>cfg.SEG_THRESHOLD, lab==3),
                ('TC', mp[1]>cfg.SEG_THRESHOLD, (lab==1)|(lab==3)),
                ('WT', mp[2]>cfg.SEG_THRESHOLD, lab>=1)]:
                pc = keep_largest_component(pred)
                res[name]['dice'].append(compute_dice(pc,true))
                res[name]['hd95'].append(compute_hd95(pc,true))
                res[name]['iou'].append(compute_iou(pc,true))
                res[name]['prec'].append(compute_precision_seg(pc,true))

```



```

        res[name]['rec'].append(compute_recall_seg(pc,true))
seg = {}
for r in ['WT','TC','ET']:
    seg[r] = {}
    for m in ['dice','hd95','iou','prec','rec']:
        vals = [x for x in res[r][m] if not np.isnan(x)]
        seg[r][m] = np.mean(vals) if vals else float('nan')
        seg[r][f'{m}_std'] = np.std(vals) if vals else 0.0
    seg[r]['n'] = len([x for x in res[r]['dice'] if not np.isnan(x)])
return seg

```

```

# — display_all —————
def display_all(train_metrics, test_metrics, seg, history):
    # 1. Training curves
    fig,ax = plt.subplots(1,2,figsize=(14,5))
    ep = range(1,len(history['train_loss'])+1)
    ax[0].plot(ep,history['train_loss'],'b-o',ms=3,label='Train')
    ax[0].plot(ep,history['val_loss'],'r-o',ms=3,label='Val')
    ax[0].set_title('Loss',fontweight='bold')
    ax[0].legend(); ax[0].grid(alpha=0.3)
    ax[1].plot(ep,history['train_acc'],'b-o',ms=3,label='Train')
    ax[1].plot(ep,history['val_acc'],'r-o',ms=3,label='Val')
    ax[1].set_title('Accuracy (IDH)',fontweight='bold')
    ax[1].legend(); ax[1].grid(alpha=0.3); ax[1].set_ylim([0,1])
    plt.suptitle('Training Curves',fontweight='bold')
    plt.tight_layout(); plt.show()

    # 2. Training confusion matrices
    nm = {'idh':'IDH\n(0=WT, 1=Mut)',
          'grade':'Grade\n(0=Low, 1=High)',
          'mgmt':'MGMT\n(0=Neg, 1=Pos)'}
    tasks = [t for t in ['idh','grade','mgmt'] if t in train_metrics]
    fig,ax = plt.subplots(1,len(tasks),figsize=(6*len(tasks),5))
    if len(tasks)==1: ax=[ax]
    for a,t in zip(ax,tasks):
        m = train_metrics[t]
        cm = confusion_matrix(m['labels'],m['preds'],labels=[0,1])
        ConfusionMatrixDisplay(cm,display_labels=['Neg','Pos']).plot(
            ax=a,cmap='Greens',colorbar=False)
        a.set_title(nm[t],fontweight='bold')
        auc_str = f"{m['auc']:.3f}" if not np.isnan(m['auc']) else 'N/A'
        a.set_xlabel(

```

```

        f"Acc={m['accuracy']:.3f}  F1={m['f1']:.3f}  AUC={auc_str}",
        fontsize=10)
plt.suptitle('Confusion Matrices – Training Set',
            fontsize=14, fontweight='bold')
plt.tight_layout(); plt.show()

# 3. Test confusion matrices
tasks = [t for t in ['idh', 'grade', 'mgmt'] if t in test_metrics]
fig, ax = plt.subplots(1, len(tasks), figsize=(6*len(tasks), 5))
if len(tasks)==1: ax=[ax]
for a, t in zip(ax, tasks):
    m = test_metrics[t]
    cm = confusion_matrix(m['labels'], m['preds'], labels=[0, 1])
    disp = ConfusionMatrixDisplay(cm, display_labels=['Neg', 'Pos'])
    disp.plot(ax=a, cmap='Blues', colorbar=False)
    a.set_title(nm[t], fontweight='bold')
    auc_str = f"{m['auc']:.3f}" if not np.isnan(m['auc']) else 'N/A'
    a.set_xlabel(
        f"Acc={m['accuracy']:.3f}  F1={m['f1']:.3f}  AUC={auc_str}",
        fontsize=10)
    unique_preds = np.unique(m['preds'])
    if len(unique_preds)==1:
        a.set_title(nm[t]+f"\n⚠ COLLAPSE (all pred={unique_preds[0]})",
                    fontweight='bold', color='red')
plt.suptitle('Confusion Matrices – Test Set',
            fontsize=14, fontweight='bold')
plt.tight_layout(); plt.show()

# 4. ROC curves
fig, a = plt.subplots(figsize=(7, 6))
col = {'idh': '#2196F3', 'grade': '#4CAF50', 'mgmt': '#FF9800'}
for t, c in col.items():
    if t not in test_metrics or np.isnan(test_metrics[t]['auc']): continue
    m = test_metrics[t]
    fpr, tpr, _ = roc_curve(m['labels'], m['probs'])
    a.plot(fpr, tpr, color=c, lw=2.5,
           label=f"{m['name']} ({m['auc']:.3f})")
a.plot([0, 1], [0, 1], 'k--', lw=1, label='Random')
a.set_title('ROC Curves – Test Set', fontweight='bold')
a.legend(); a.grid(alpha=0.3)
a.set_xlabel('FPR'); a.set_ylabel('TPR')
plt.tight_layout(); plt.show()

```

```

# 5. Segmentation bars (Dice + HD95)
regions=['WT','TC','ET']; cols=['#2196F3','#4CAF50','#FF9800']
dv=[seg[r]['dice'] for r in regions]
hv=[seg[r]['hd95'] for r in regions]
fig,ax = plt.subplots(1,2,figsize=(12,5))
ax[0].bar(regions,dv,color=cols,
          yerr=[seg[r]['dice_std'] for r in regions],
          capsize=5,edgecolor='black',lw=0.5)
ax[0].set_ylim([0,1.05])
ax[0].set_title('Dice (higher better)',fontweight='bold')
ax[0].axhline(0.8,color='r',ls='--',alpha=0.5)
ax[0].grid(alpha=0.3,axis='y')
for i,v in enumerate(dv):
    if not np.isnan(v):
        ax[0].text(i,v+0.03,f'{v:.3f}',ha='center',fontweight='bold')
hv2=[h if not np.isnan(h) else 0 for h in hv]
ax[1].bar(regions,hv2,color=cols,capsize=5,edgecolor='black',lw=0.5)
ax[1].set_title('HD95 mm (lower better)',fontweight='bold')
ax[1].grid(alpha=0.3,axis='y')
for i,v in enumerate(hv):
    if not np.isnan(v):
        ax[1].text(i,hv2[i]+0.5,f'{v:.1f}',ha='center',fontweight='bold')
plt.suptitle('SwinUNETR Segmentation – Test Set',fontweight='bold')
plt.tight_layout(); plt.show()

# 6. Text summary
print("\n"+"="*55+"\n  SUMMARY\n"+"="*55)
print("\nTRAINING Classification:")
for t in ['idh','grade','mgmt']:
    if t not in train_metrics: continue
    m = train_metrics[t]
    auc = f"{m['auc']:.3f}" if not np.isnan(m['auc']) else 'N/A'
    print(f"\n  {m['name']}")
    print(f"    Accuracy : {m['accuracy']:.3f}")
    print(f"    Precision : {m['precision']:.3f}")
    print(f"    Recall    : {m['recall']:.3f}")
    print(f"    F1        : {m['f1']:.3f}")
    print(f"    AUC       : {auc}")
    print(f"    Sens/Spec : {m['sensitivity']:.3f} / {m['specificity']:.3f}")
    print(f"    TP={m['tp']}  FP={m['fp']}  TN={m['tn']}  FN={m['fn']}")

```

```

print("\nTEST Classification:")
for t in ['idh','grade','mgmt']:
    if t not in test_metrics: continue
    m = test_metrics[t]
    auc = f"{m['auc']:.3f}" if not np.isnan(m['auc']) else 'N/A'
    print(f"\n  {m['name']}")
    print(f"    Accuracy  : {m['accuracy']:.3f}")
    print(f"    Precision : {m['precision']:.3f}")
    print(f"    Recall    : {m['recall']:.3f}")
    print(f"    F1        : {m['f1']:.3f}")
    print(f"    AUC       : {auc}")
    print(f"    Sens/Spec : {m['sensitivity']:.3f} / {m['specificity']:.3f}")
    print(f"    TP={m['tp']} FP={m['fp']} TN={m['tn']} FN={m['fn']}")

print("\nSegmentation:")
for r in ['WT','TC','ET']:
    m = seg[r]
    print(f"  {r}")
    print(f"    Dice      : {m['dice']:.3f} ± {m['dice_std']:.3f}")
    print(f"    IoU       : {m['iou']:.3f} ± {m['iou_std']:.3f}")
    print(f"    HD95 (mm) : {m['hd95']:.1f} ± {m['hd95_std']:.1f}")
    print(f"    Precision : {m['prec']:.3f} ± {m['prec_std']:.3f}")
    print(f"    Recall    : {m['rec']:.3f} ± {m['rec_std']:.3f}")
print("="*55)

```

MULTI SEED

```

import random
import numpy as np
import pandas as pd

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

```

```
SEEDS = [0, 1, 2, 3, 4]
```

```

all_results = []

for seed in SEEDS:
    print(f"\n{' '*55}\n SEED {seed}\n{' '*55}")
    set_seed(seed)

    classifier = BrainTumorClassifier().to(cfg.DEVICE)
    classifier, history = train_classifier(
        classifier, SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR, tag=f"seed{seed}")

    _, _, test_loader = build_loaders(SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)
    metrics = evaluate_classifier(classifier, test_loader)

    row = {'seed': seed}
    for task in ['idh', 'grade', 'mgmt']:
        if task in metrics:
            row[f'{task}_auc'] = metrics[task]['auc']
            row[f'{task}_acc'] = metrics[task]['accuracy']
            row[f'{task}_f1'] = metrics[task]['f1']
            row[f'{task}_sens'] = metrics[task]['sensitivity']
            row[f'{task}_spec'] = metrics[task]['specificity']
    all_results.append(row)

results_df = pd.DataFrame(all_results)
print("\n\nPer-seed results:")
print(results_df.round(3))

print(f"\n{' '*55}\n SUMMARY (mean  $\pm$  std across {len(SEEDS)} seeds)\n{' '*55}")
for task in ['idh', 'grade', 'mgmt']:
    col = f'{task}_auc'
    if col in results_df.columns:
        m, s = results_df[col].mean(), results_df[col].std()
        lo, hi = results_df[col].min(), results_df[col].max()
        print(f"{task.upper():6s} AUC: {m:.3f}  $\pm$  {s:.3f} (range {lo:.3f}–{hi:.3f})")

results_df.to_csv(
    '/content/drive/MyDrive/checkpoints/cv_results_df.csv', index=False)
print("results_df saved to Drive")

```

```

=====
SEED 0
=====

```

Patients loaded: 487

Patients skipped: 14

IDH: 391 wt / 96 mut

Grade: 92 low / 395 high

MGMT: 112 neg / 297 pos

Patients with cache: 487

Split: 343 train / 72 val / 72 test

Training variant seed0 from scratch

Ep 1 | Train 1.2331 acc 0.784 | Val 0.8846 acc 0.903

✓ Best saved (val 0.8846)

Ep 2 | Train 1.0619 acc 0.854 | Val 0.8143 acc 0.903

✓ Best saved (val 0.8143)

Ep 3 | Train 1.0441 acc 0.845 | Val 0.9386 acc 0.847

Ep 4 | Train 1.0007 acc 0.837 | Val 0.8311 acc 0.917

Ep 5 | Train 1.0193 acc 0.825 | Val 0.8562 acc 0.903

Ep 6 | Train 1.0557 acc 0.831 | Val 0.8391 acc 0.889

Ep 7 | Train 0.9970 acc 0.822 | Val 0.8084 acc 0.903

✓ Best saved (val 0.8084)

Ep 8 | Train 1.0036 acc 0.831 | Val 0.7998 acc 0.903

✓ Best saved (val 0.7998)

Ep 9 | Train 0.9887 acc 0.843 | Val 0.8440 acc 0.903

Ep 10 | Train 1.0241 acc 0.816 | Val 0.7898 acc 0.903

✓ Best saved (val 0.7898)

Ep 11 | Train 0.9593 acc 0.822 | Val 0.8389 acc 0.875

Ep 12 | Train 0.9784 acc 0.834 | Val 0.8544 acc 0.903

Ep 13 | Train 0.9805 acc 0.848 | Val 0.8257 acc 0.903

Ep 14 | Train 1.0104 acc 0.843 | Val 0.9161 acc 0.819

Ep 15 | Train 0.9754 acc 0.851 | Val 0.7804 acc 0.903

✓ Best saved (val 0.7804)

Ep 16 | Train 0.9335 acc 0.860 | Val 0.7560 acc 0.931

✓ Best saved (val 0.7560)

Ep 17 | Train 0.9577 acc 0.845 | Val 0.8357 acc 0.889

Ep 18 | Train 0.9706 acc 0.840 | Val 0.7916 acc 0.917

Ep 19 | Train 0.9666 acc 0.860 | Val 0.7697 acc 0.903

Ep 20 | Train 0.9458 acc 0.860 | Val 0.8008 acc 0.903

Ep 21 | Train 0.9075 acc 0.863 | Val 0.7632 acc 0.917

Ep 22 | Train 0.9247 acc 0.872 | Val 0.8490 acc 0.806

Ep 23 | Train 0.9458 acc 0.845 | Val 0.7955 acc 0.931

Ep 24 | Train 0.9095 acc 0.872 | Val 0.7849 acc 0.917

Ep 25 | Train 0.9218 acc 0.863 | Val 0.7642 acc 0.931

Ep 26 | Train 0.9065 acc 0.883 | Val 0.8183 acc 0.875

Ep 27 | Train 0.9057 acc 0.869 | Val 0.7801 acc 0.917

Ep 28 | Train 0.8983 acc 0.883 | Val 0.7762 acc 0.917

Ep 29 | Train 0.9039 acc 0.869 | Val 0.7536 acc 0.917

✓ Best saved (val 0.7536)

Ep 30	Train 0.8700	acc 0.883	Val 0.7553	acc 0.931
Ep 31	Train 0.9229	acc 0.869	Val 0.7546	acc 0.931
Ep 32	Train 0.8702	acc 0.866	Val 0.7658	acc 0.931
Ep 33	Train 0.8739	acc 0.872	Val 0.7990	acc 0.875
Ep 34	Train 0.8815	acc 0.872	Val 0.7601	acc 0.903
Ep 35	Train 0.9072	acc 0.892	Val 0.7680	acc 0.931
Ep 36	Train 0.8423	acc 0.895	Val 0.7642	acc 0.889

```
# CELL 15B
import pandas as pd, random

# — Load results_df if not in memory —————
if 'results_df' not in dir():
    csv_path = '/content/drive/MyDrive/checkpoints/cv_results_df.csv'
    results_df = pd.read_csv(csv_path)
    SEEDS = [0, 1, 2, 3, 4]
    print("Loaded results_df from Drive")

# — Sanity check —————
CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_ucsf_finetuned'
sample_file = random.choice(
    [f for f in os.listdir(CACHE_DIR) if f.endswith('.pt')])
d = torch.load(os.path.join(CACHE_DIR, sample_file),
               map_location='cpu', weights_only=False)
mp = d['mean_pred'].numpy()
lab = d['label'].squeeze().numpy()
wt_pred = mp[2] > 0.3
wt_true = lab >= 1
u = wt_pred.sum() + wt_true.sum()
dice = 2*(wt_pred & wt_true).sum()/u if u > 0 else 1.0
print(f"Sample {sample_file} – WT Dice: {dice:.4f}")

# — Segmentation eval (once only) —————
seg = evaluate_segmentation(CACHE_DIR, cfg.UCSF_CSV, cfg.UCSF_DIR)

# — Aggregate classification summary —————
print("="*55)
print(f" AGGREGATE CLASSIFICATION SUMMARY (mean ± std, n={len(SEEDS)} seeds)")
print("="*55)
for task in ['idh', 'grade', 'mgmt']:
    col = f'{task}_auc'
    if col in results_df.columns:
```

```

print(f"\n{task.upper()}")
for metric in ['auc', 'acc', 'f1', 'sens', 'spec']:
    c = f'{task}_{metric}'
    m, s = results_df[c].mean(), results_df[c].std()
    print(f" {metric:<12}: {m:.3f} ± {s:.3f}")

print("\nSegmentation:")
for r in ['WT', 'TC', 'ET']:
    m = seg[r]
    print(f" {r} Dice={m['dice']:.3f} IoU={m['iou']:.3f} "
          f"HD95={m['hd95']:.1f} Prec={m['prec']:.3f} Rec={m['rec']:.3f}")
print("="*55)

# — Pick representative seed for plots —————
rep_seed = results_df.iloc[
    (results_df['idh_auc'] - results_df['idh_auc'].mean()).abs().argsort()[0:1]
]['seed'].values[0]
print(f"\nPlots from seed {rep_seed} (closest to mean IDH AUC)")

ckpt = torch.load(os.path.join(cfg.CKPT_DIR, f'clf_best_seed{rep_seed}.pth'),
                  map_location=cfg.DEVICE, weights_only=False)
classifier.load_state_dict(ckpt['model_state'])
latest = torch.load(os.path.join(cfg.CKPT_DIR, f'clf_latest_seed{rep_seed}.pth'),
                   map_location=cfg.DEVICE, weights_only=False)

# — Build loaders for train and test —————
train_loader_eval, _, test_loader = build_loaders(
    SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)

# — Evaluate on train and test —————
print("\nEvaluating on training set...")
train_metrics = evaluate_classifier(classifier, train_loader_eval)

print("Evaluating on test set...")
test_metrics = evaluate_classifier(classifier, test_loader)

# — Display everything —————
display_all(train_metrics, test_metrics, seg, latest['history'])

```




Sample UCSF-PDGM-174.pt – WT Dice: 0.8694

Patients loaded: 487

Patients skipped: 14

IDH: 391 wt / 96 mut

Grade: 92 low / 395 high

MGMT: 112 neg / 297 pos

Segmentation eval: 100%|██████████| 72/72 [01:57<00:00, 1.63s/it]

=====

AGGREGATE CLASSIFICATION SUMMARY (mean ± std, n=5 seeds)

=====

```
# =====
# CELL 15C – McNemar's Statistical Significance Test
# =====

!pip install -q statsmodels
from statsmodels.stats.contingency_tables import mcnemar

def run_mcnemar(metrics, split_name="Test"):
    """
    Compares model predictions against a majority-class baseline
    using McNemar's test. Reports p-value and interpretation.
    Requires test_metrics or metrics_rep from Cell 15.
    """
    # Majority class for each task based on UCSF training distribution
    majority = {'idh': 0, 'grade': 1, 'mgmt': 1}
    task_names = {'idh': 'IDH Status', 'grade': 'WHO Grade', 'mgmt': 'MGMT'}

    print(f"\n{'='*55}")
    print(f"  McNEMAR'S TEST – {split_name} Set")
    print(f"  (Model vs. Majority-Class Baseline)")
    print(f"{'='*55}")

    for task in ['idh', 'grade', 'mgmt']:
        if task not in metrics:
            continue
        m = metrics[task]
        model_preds = m['preds']          # already binary (0/1)
        true_labels = m['labels'].astype(int)
        maj_class = majority[task]
        baseline_preds = np.full_like(true_labels, maj_class)

        # Discordant pairs
        # b = model right, baseline wrong
```

```

# c = model wrong, baseline right
b = int(np.sum((model_preds == true_labels) &
               (baseline_preds != true_labels)))
c = int(np.sum((model_preds != true_labels) &
               (baseline_preds == true_labels)))

if b + c == 0:
    print(f"\n{task_names[task]}")
    print(f" No discordant pairs – test not applicable")
    continue

table = [[0, b], [c, 0]]
result = mcnemar(table, exact=True)
pval = result.pvalue

# Baseline accuracy for reference
baseline_acc = np.mean(baseline_preds == true_labels)
model_acc = m['accuracy']

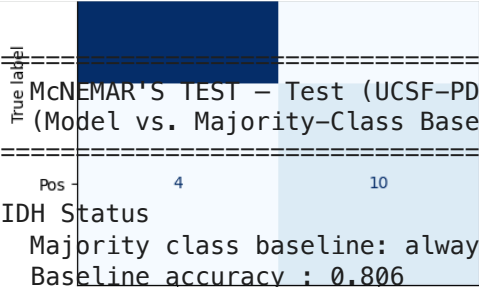
print(f"\n{task_names[task]}")
print(f" Majority class baseline: always predict {maj_class} "
      f"('{WT}' if task=='idh' and maj_class==0 else 'High' if task=='grade' and maj_class==1 else 'Pos' if maj_class")
print(f" Baseline accuracy : {baseline_acc:.3f}")
print(f" Model accuracy : {model_acc:.3f}")
print(f" Discordant pairs : b={b} (model✓ basex) c={c} (modelx base✓)")
print(f" p-value : {pval:.4f}", end="")
if pval < 0.001:
    print(" *** (p<0.001, highly significant)")
elif pval < 0.01:
    print(" ** (p<0.01, significant)")
elif pval < 0.05:
    print(" * (p<0.05, significant)")
else:
    print(f" (p≥0.05, not significant vs. baseline)")

print(f"\n{'='*55}")
print("Note: McNemar's test (exact) compares model against")
print("majority-class baseline on matched predictions.")
print("Significant p-value = model outperforms naive baseline.")
print(f"{'='*55}")

```

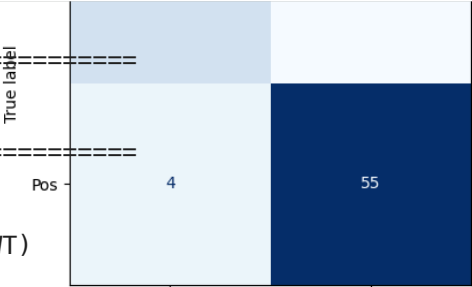
```
# Run on test set – uses metrics_rep from Cell 15
# (the representative seed closest to mean IDH AUC)
run_mcnemar(test_metrics, split_name="Test (UCSF-PDGM)")

# Optionally also run on training set to show no overfitting
run_mcnemar(train_metrics, split_name="Train (UCSF-PDGM)")
```



McNEMAR'S TEST – Test (UCSF-PDGM) Set
(Model vs. Majority-Class Baseline)

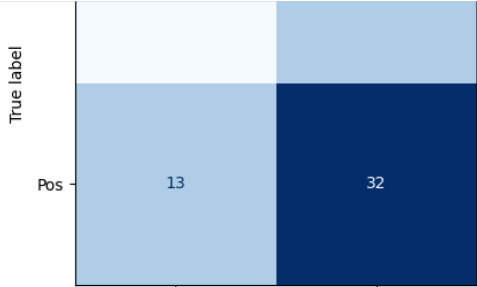
IDH Status
Majority class baseline: always predict 0 (WT)
Baseline accuracy : 0.806
Model accuracy : 0.889
Discordant pairs : b=10 (model vs. base) c=4 (model vs. base)
p-value : 0.1796 (p>0.05, not significant vs. baseline)



True label

Pos

Acc=0.917 F1=0.948 AUC=0.975



True label

Pos

Acc=0.581 F1=0.711 AUC=0.431

ROC curves – Test Set

WHO Grade
Majority class baseline: always predict 1 (High)
Baseline accuracy : 0.819
Model accuracy : 0.917
Discordant pairs : b=11 (model vs. base) c=4 (model vs. base)
p-value : 0.1185 (p>0.05, not significant vs. baseline)

MGMT
Majority class baseline: always predict 1 (Pos)
Baseline accuracy : 0.726
Model accuracy : 0.581
Discordant pairs : b=4 (model vs. base) c=13 (model vs. base)
p-value : 0.0490 * (p<0.05, significant)

Note: McNemar's test (exact) compares model against majority-class baseline on matched predictions.
Significant p-value = model outperforms naive baseline.

McNEMAR'S TEST – Train (UCSF-PDGM) Set
(Model vs. Majority-Class Baseline)

IDH Status

IDH Status (0.910)
WHO Grade (0.975)

Majority class baseline: always predict 0 (WT)

Baseline accuracy : 0.802

Model accuracy : 0.880

Discordant pairs : b=24 (model $\checkmark$ base $\times$) c=27 (model $\times$ base $\checkmark$) 0.8

p-value : 0.0036 ** (p<0.01, significant)

MGMT (0.431)
---- Random

WHO Grade

Majority class baseline: always predict 1 (High)

Baseline accuracy : 0.48

Model accuracy : 0.913

Discordant pairs : b=62 (model $\checkmark$ base $\times$) c=26 (model $\times$ base $\checkmark$)

p-value : 0.0002 *** (p<0.001, highly significant)

MGMT

Majority class baseline: always predict 1 (Pos)

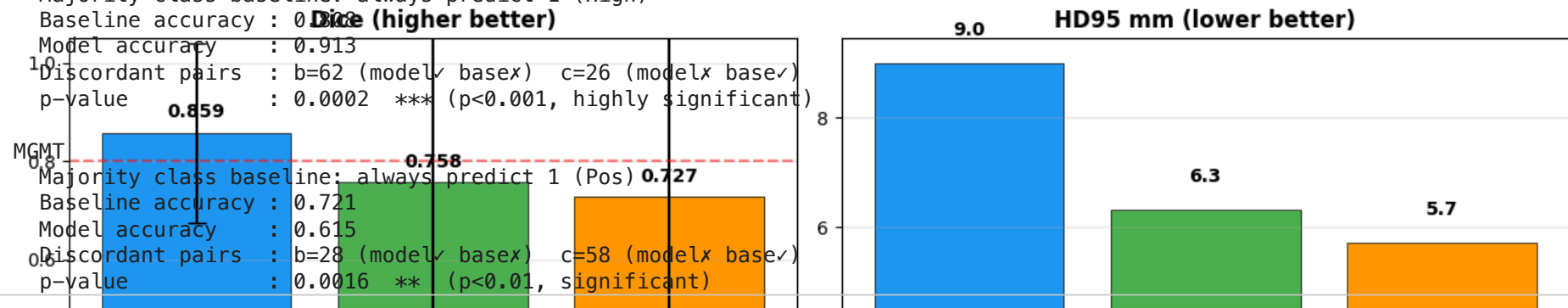
Baseline accuracy : 0.721

Model accuracy : 0.615

Discordant pairs : b=28 (model $\checkmark$ base $\times$) c=58 (model $\times$ base $\checkmark$)

p-value : 0.0016 ** (p<0.01, significant)

SwinUNETR Segmentation — Test Set



```
# =====
# CELL 13B – Five-Fold Cross Validation
# =====

from sklearn.model_selection import StratifiedKFold

def train_classifier_fold(clf_model, train_loader, val_loader,
                        epochs=cfg.CLF_EPOCHS, patience=20, tag='fold'):
    """
    CV-specific trainer that accepts pre-built loaders directly
    instead of rebuilding them from scratch – avoids the fixed-split bug.
    """
    optimizer = torch.optim.AdamW(
        clf_model.parameters(), lr=cfg.LR, weight_decay=1e-5)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=epochs)
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val, no_improve = float('inf'), 0
    best_state = None

    for epoch in range(epochs):
        clf_model.train()
        ep_loss, ep_corr, ep_tot = 0.0, 0, 0
        for batch in train_loader:
            enc = batch['encoder'].to(cfg.DEVICE)
```

```

mor = batch['morph'].to(cfg.DEVICE)
conf = batch['conf_score'].to(cfg.DEVICE)
lbl = {k: batch[k].to(cfg.DEVICE)
        for k in ['idh', 'grade', 'mgmt']}
optimizer.zero_grad()
preds = clf_model(enc, mor, conf)
loss = classification_loss(preds, lbl)
loss.backward()
optimizer.step()
ep_loss += loss.item()
v = ~torch.isnan(lbl['idh'])
if v.any():
    p = torch.sigmoid(preds['idh'][v])>0.5
    t = lbl['idh'][v]>0.5
    ep_corr += (p==t).sum().item()
    ep_tot += v.sum().item()
scheduler.step()
tr_loss = ep_loss/len(train_loader)
tr_acc = ep_corr/ep_tot if ep_tot>0 else 0.0

clf_model.eval()
v_loss, v_corr, v_tot = 0.0, 0, 0
with torch.no_grad():
    for batch in val_loader:
        enc = batch['encoder'].to(cfg.DEVICE)
        mor = batch['morph'].to(cfg.DEVICE)
        conf = batch['conf_score'].to(cfg.DEVICE)
        lbl = {k: batch[k].to(cfg.DEVICE)
                for k in ['idh', 'grade', 'mgmt']}
        preds = clf_model(enc, mor, conf)
        v_loss += classification_loss(preds, lbl).item()
        vv = ~torch.isnan(lbl['idh'])
        if vv.any():
            p = torch.sigmoid(preds['idh'][vv])>0.5
            t = lbl['idh'][vv]>0.5
            v_corr += (p==t).sum().item()
            v_tot += vv.sum().item()
va_loss = v_loss/len(val_loader)
va_acc = v_corr/v_tot if v_tot>0 else 0.0
history['train_loss'].append(tr_loss)
history['val_loss'].append(va_loss)
history['train_acc'].append(tr_acc)

```

```

history['val_acc'].append(va_acc)

if va_loss < best_val:
    best_val = va_loss
    no_improve = 0
    best_state = {k:v.cpu().clone()
                  for k,v in clf_model.state_dict().items()}
else:
    no_improve += 1
    if no_improve >= patience:
        break

# Restore best weights
if best_state:
    clf_model.load_state_dict(
        {k:v.to(cfg.DEVICE) for k,v in best_state.items()})
return clf_model, history

def run_five_fold_cv(cache_dir, csv_path, ucsf_dir, seed=42):
    all_dicts = build_ucsf_dicts(csv_path, ucsf_dir)
    cached_pids = {f.replace('.pt','') for f in os.listdir(cache_dir)
                    if f.endswith('.pt')}
    all_dicts = [d for d in all_dicts if d['patient_id'] in cached_pids]
    print(f"Total patients for CV: {len(all_dicts)}")

    # Stratify on IDH for balanced folds
    labels = np.array([0.0 if d['idh']==0.0
                       else 1.0 if d['idh']==1.0
                       else -1.0 for d in all_dicts])
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)

    fold_results = []
    all_probs = {t:[] for t in ['idh','grade','mgmt']}
    all_labels_cv= {t:[] for t in ['idh','grade','mgmt']}

    for fold, (train_val_idx, test_idx) in enumerate(
        skf.split(all_dicts, labels)):
        print(f"\n{' '*55}\n  FOLD {fold+1}/5\n{' '*55}")
        set_seed(seed + fold)

        # 85% train, 15% val from the non-test portion

```

```

train_val = [all_dicts[i] for i in train_val_idx]
test_d     = [all_dicts[i] for i in test_idx]
n_val      = max(1, int(len(train_val) * 0.15))
val_d      = train_val[:n_val]
train_d    = train_val[n_val:]

print(f" Train: {len(train_d)} Val: {len(val_d)} Test: {len(test_d)}")

def get_paths(dicts):
    return [os.path.join(cache_dir, f"{d['patient_id']}.pt")
            for d in dicts]

train_loader = torch.utils.data.DataLoader(
    CachedDataset(get_paths(train_d)),
    batch_size=cfg.BATCH_SIZE, shuffle=True, num_workers=0)
val_loader = torch.utils.data.DataLoader(
    CachedDataset(get_paths(val_d)),
    batch_size=cfg.BATCH_SIZE, shuffle=False, num_workers=0)
test_loader = torch.utils.data.DataLoader(
    CachedDataset(get_paths(test_d)),
    batch_size=1, shuffle=False, num_workers=0)

clf = BrainTumorClassifier().to(cfg.DEVICE)
clf, _ = train_classifier_fold(
    clf, train_loader, val_loader, tag=f"fold{fold+1}")

metrics = evaluate_classifier(clf, test_loader)
fold_results.append(metrics)

for t in ['idh', 'grade', 'mgmt']:
    if t in metrics:
        all_probs[t].extend(metrics[t]['probs'].tolist())
        all_labels_cv[t].extend(metrics[t]['labels'].tolist())

# — Summary —————
print(f"\n{'='*55}\n 5-FOLD CV SUMMARY\n{'='*55}")
for task in ['idh', 'grade', 'mgmt']:
    aucs = [r[task]['auc'] for r in fold_results
            if task in r and not np.isnan(r[task]['auc'])]
    f1s = [r[task]['f1'] for r in fold_results if task in r]
    accs = [r[task]['accuracy'] for r in fold_results if task in r]
    if not aucs:

```



```

        continue
    print(f"\n{task.upper()}")
    print(f"   AUC : {np.mean(aucs):.3f} ± {np.std(aucs):.3f}   "
          f"(range {min(aucs):.3f}–{max(aucs):.3f})")
    print(f"   F1  : {np.mean(f1s):.3f} ± {np.std(f1s):.3f}")
    print(f"   Acc : {np.mean(accs):.3f} ± {np.std(accs):.3f}")

# Pooled ROC across all folds
print(f"\nPooled AUC (predictions from all 5 folds combined):")
for task in ['idh', 'grade', 'mgmt']:
    p = np.array(all_probs[task])
    l = np.array(all_labels_cv[task])
    valid = ~np.isnan(l)
    if valid.sum()==0: continue
    try:
        pooled_auc = roc_auc_score(l[valid], p[valid])
        print(f"  {task.upper()}: {pooled_auc:.3f}  (n={valid.sum()})")
    except:
        print(f"  {task.upper()}: N/A")

return fold_results, all_probs, all_labels_cv

# Run – expect ~20–25 minutes total (5 folds × ~4 min each)
cv_results, cv_probs, cv_labels = run_five_fold_cv(
    SLIM_CACHE, cfg.UCSF_CSV, cfg.UCSF_DIR)

```

Patients loaded: 487
 Patients skipped: 14

IDH: 391 wt / 96 mut
 Grade: 92 low / 395 high
 MGMT: 112 neg / 297 pos
 Total patients for CV: 487

```

=====
FOLD 1/5
=====
Train: 331 Val: 58 Test: 98
Classification eval: 100%|██████████| 98/98 [00:00<00:00, 235.56it/s]

```

```

=====
FOLD 2/5
=====
Train: 331 Val: 58 Test: 98

```

```
Classification eval: 100%|██████████| 98/98 [00:00<00:00, 216.71it/s]
```

```
=====
FOLD 3/5
=====
```

```
Train: 332 Val: 58 Test: 97
Classification eval: 100%|██████████| 97/97 [00:00<00:00, 240.21it/s]
```

```
=====
FOLD 4/5
=====
```

```
Train: 332 Val: 58 Test: 97
Classification eval: 100%|██████████| 97/97 [00:00<00:00, 203.05it/s]
```

```
=====
FOLD 5/5
=====
```

```
Train: 332 Val: 58 Test: 97
Classification eval: 100%|██████████| 97/97 [00:00<00:00, 223.89it/s]
```

```
=====
5-FOLD CV SUMMARY
=====
```

```
IDH
```

```
AUC : 0.906 ± 0.055 (range 0.810–0.977)
F1 : 0.721 ± 0.079
Acc : 0.885 ± 0.030
```

```
GRADE
```

```
AUC : 0.980 ± 0.011 (range 0.965–0.996)
F1 : 0.960 ± 0.010
Acc : 0.936 ± 0.014
```

```
MGMT
```

```
AUC : 0.519 ± 0.048 (range 0.469–0.599)
F1 : 0.565 ± 0.190
Acc : 0.511 ± 0.125
```

```
Pooled AUC (predictions from all 5 folds combined):
IDH: 0.902 (n=487)
```

```
# =====
# CELL 15B – UTSW Cross-dataset evaluation
# =====
```

```
def evaluate_segmentation_utsw(cache_dir):
```

```

"""Evaluate segmentation on ALL UTSW patients (no train/val split –
this is pure zero-shot cross-dataset evaluation)."""
files = [f for f in os.listdir(cache_dir) if f.endswith('.pt')]
res = {r: {'dice': [], 'hd95': []} for r in ['ET', 'TC', 'WT']}

for fname in tqdm(files, desc="UTSW seg eval"):
    try:
        d = torch.load(os.path.join(cache_dir, fname),
                        map_location='cpu', weights_only=False)
    except: continue
    if 'mean_pred' not in d: continue
    mp = d['mean_pred'].numpy()
    lab = d['label'].squeeze().numpy()
    for name, pred, true in [
        ('ET', mp[0]>cfg.SEG_THRESHOLD, lab==3),
        ('TC', mp[1]>cfg.SEG_THRESHOLD, (lab==1)|(lab==3)),
        ('WT', mp[2]>cfg.SEG_THRESHOLD, lab>=1)]:
        pred_clean = keep_largest_component(pred)
        res[name]['dice'].append(compute_dice(pred_clean, true))
        res[name]['hd95'].append(compute_hd95(pred_clean, true))

print("\n=== UTSW Cross-Dataset Segmentation (zero-shot) ===")
for r in ['WT', 'TC', 'ET']:
    dv = [x for x in res[r]['dice'] if not np.isnan(x)]
    hv = [x for x in res[r]['hd95'] if not np.isnan(x)]
    print(f" {r}: Dice={np.mean(dv):.3f}±{np.std(dv):.3f} "
          f"HD95={np.mean(hv):.1f}±{np.std(hv):.1f} (n={len(dv)})")
return res

def evaluate_classifier_utsw(clf_model, cache_dir):
    """Zero-shot classification on UTSW using UCSF-trained classifier."""
    files = [f for f in os.listdir(cache_dir) if f.endswith('.pt')]
    res = {t: {'probs': [], 'labels': []} for t in ['idh', 'grade', 'mgmt']}

    clf_model.eval()
    with torch.no_grad():
        for fname in tqdm(files, desc="UTSW clf eval"):
            try:
                d = torch.load(os.path.join(cache_dir, fname),
                                map_location='cpu', weights_only=False)
            except: continue
            enc = d['encoder'].unsqueeze(0).to(cfg.DEVICE)

```

```

mor = d['morph'].unsqueeze(0).to(cfg.DEVICE)
conf = d['conf_score'].view(1,1).to(cfg.DEVICE)
preds = clf_model(enc, mor, conf)
for t in ['idh', 'grade', 'mgmt']:
    res[t]['probs'].append(
        torch.sigmoid(preds[t]).item())
    res[t]['labels'].append(d[t].item())

print("\n=== UTSW Cross-Dataset Classification (zero-shot) ===")
for t in ['idh', 'grade', 'mgmt']:
    p = np.array(res[t]['probs'])
    l = np.array(res[t]['labels'])
    valid = ~np.isnan(l)
    p, l = p[valid], l[valid]
    if len(l) == 0:
        print(f" {t.upper()}: no valid labels")
        continue
    try:
        auc = roc_auc_score(l, p)
    except:
        auc = float('nan')
    pb = (p >= 0.5).astype(int)
    acc = accuracy_score(l, pb)
    f1 = f1_score(l, pb, zero_division=0)
    print(f" {t.upper()}: AUC={auc:.3f} Acc={acc:.3f} F1={f1:.3f} (n={len(l)})")

# — Run UTSW evaluation —————
UTSW_CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_utsw'
n_utsw = len([f for f in os.listdir(UTSW_CACHE_DIR) if f.endswith('.pt')])
print(f"UTSW cache files: {n_utsw}")

utsw_seg = evaluate_segmentation_utsw(UTSW_CACHE_DIR)

# Load best classifier (use seed 1 representative, or whichever seed you reported)
ckpt = torch.load(os.path.join(cfg.CKPT_DIR, 'clf_best_seed1.pth'),
                  map_location=cfg.DEVICE, weights_only=False)
classifier.load_state_dict(ckpt['model_state'])
evaluate_classifier_utsw(classifier, UTSW_CACHE_DIR)

```

```

# =====
# CELL 15D – BraTS 2023 GLI cross-dataset segmentation evaluation

```

```

# =====
def evaluate_segmentation_brats23(cache_dir):
    files = [f for f in os.listdir(cache_dir) if f.endswith('.pt')]
    print(f"Evaluating {len(files)} BraTS 2023 patients...")
    res = {r: {'dice': [], 'hd95': [], 'iou': [], 'prec': [], 'rec': []}
           for r in ['ET', 'TC', 'WT']}

    for fname in tqdm(files, desc="BraTS23 seg eval"):
        try:
            d = torch.load(os.path.join(cache_dir, fname),
                           map_location='cpu', weights_only=False)
        except: continue
        if 'mean_pred' not in d: continue
        mp = d['mean_pred'].numpy()
        lab = d['label'].squeeze().numpy()

        for name, pred, true in [
            ('ET', mp[0]>cfg.SEG_THRESHOLD, lab==3),
            ('TC', mp[1]>cfg.SEG_THRESHOLD, (lab==1)|(lab==3)),
            ('WT', mp[2]>cfg.SEG_THRESHOLD, lab>=1)]:
            pc = keep_largest_component(pred)
            res[name]['dice'].append(compute_dice(pc, true))
            res[name]['hd95'].append(compute_hd95(pc, true))
            res[name]['iou'].append(compute_iou(pc, true))
            res[name]['prec'].append(compute_precision_seg(pc, true))
            res[name]['rec'].append(compute_recall_seg(pc, true))

    print("\n=== BraTS 2023 GLI Cross-Dataset Segmentation (zero-shot) ===")
    for r in ['WT', 'TC', 'ET']:
        dv = [x for x in res[r]['dice'] if not np.isnan(x)]
        hv = [x for x in res[r]['hd95'] if not np.isnan(x)]
        iv = [x for x in res[r]['iou'] if not np.isnan(x)]
        pv = [x for x in res[r]['prec'] if not np.isnan(x)]
        rv = [x for x in res[r]['rec'] if not np.isnan(x)]
        print(f"\n {r} (n={len(dv)})")
        print(f"    Dice : {np.mean(dv):.3f} ± {np.std(dv):.3f}")
        print(f"    IoU  : {np.mean(iv):.3f} ± {np.std(iv):.3f}")
        print(f"    HD95 : {np.mean(hv):.1f} ± {np.std(hv):.1f} mm")
        print(f"    Prec : {np.mean(pv):.3f} ± {np.std(pv):.3f}")
        print(f"    Rec  : {np.mean(rv):.3f} ± {np.std(rv):.3f}")
    return res

```

```
BRATS23_CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_brats23'
n = len([f for f in os.listdir(BRATS23_CACHE_DIR) if f.endswith('.pt')])
print(f"Brats23 cache files: {n}")
brats23_seg = evaluate_segmentation_brats23(BRATS23_CACHE_DIR)
```

```
Brats23 cache files: 1251
Evaluating 1251 Brats 2023 patients...
Brats23 seg eval: 1%|          | 12/1251 [00:31<52:17, 2.53s/it]
```

```
def evaluate_segmentation_brats23(cache_dir):
    files = [f for f in os.listdir(cache_dir) if f.endswith('.pt')]
    print(f"Evaluating {len(files)} Brats 2023 patients...")
    res = {r: {'dice': [], 'iou': [], 'prec': [], 'rec': []}
           for r in ['ET', 'TC', 'WT']}
    failed_count = 0

    for fname in tqdm(files, desc="Brats23 seg eval"):
        try:
            d = torch.load(os.path.join(cache_dir, fname),
                           map_location='cpu', weights_only=False)
            if 'mean_pred' not in d:
                continue
            mp = d['mean_pred'].numpy()
            lab = d['label'].squeeze().numpy()

            for name, pred, true in [
                ('ET', mp[0]>0.3, lab==3),
                ('TC', mp[1]>0.3, (lab==1)|(lab==3)),
                ('WT', mp[2]>0.3, lab>=1)]:
                pc = keep_largest_component(pred)
                res[name]['dice'].append(compute_dice(pc, true))
                res[name]['iou'].append(compute_iou(pc, true))
                res[name]['prec'].append(compute_precision_seg(pc, true))
                res[name]['rec'].append(compute_recall_seg(pc, true))

        except Exception as e:
            failed_count += 1
            if failed_count <= 5: # Print first 5 errors only
                print(f"Failed {fname}: {type(e).__name__}: {e}")
            continue

    print(f"\nFailed: {failed_count}/{len(files)}")
```

```

print( '\n=== BraTS 2023 GLI Cross-Dataset Segmentation (zero-shot) === ')
for r in ['WT', 'TC', 'ET']:
    dv = [x for x in res[r]['dice'] if not np.isnan(x)]
    iv = [x for x in res[r]['iou'] if not np.isnan(x)]
    pv = [x for x in res[r]['prec'] if not np.isnan(x)]
    rv = [x for x in res[r]['rec'] if not np.isnan(x)]
    print(f"\n {r} (n={len(dv)})")
    print(f"    Dice : {np.mean(dv):.3f} ± {np.std(dv):.3f}")
    print(f"    IoU  : {np.mean(iv):.3f} ± {np.std(iv):.3f}")
    print(f"    Prec : {np.mean(pv):.3f} ± {np.std(pv):.3f}")
    print(f"    Rec  : {np.mean(rv):.3f} ± {np.std(rv):.3f}")
return res

```

```

BRATS23_CACHE_DIR = '/content/drive/MyDrive/checkpoints/feature_cache_3d_brats23'
brats23_seg = evaluate_segmentation_brats23(BRATS23_CACHE_DIR)

```

Evaluating 1251 BraTS 2023 patients...

BraTS23 seg eval: 100%|██████████| 1251/1251 [26:19<00:00, 1.26s/it]

Failed: 0/1251

=== BraTS 2023 GLI Cross-Dataset Segmentation (zero-shot) ===

WT (n=1251)

Dice : 0.786 ± 0.235

IoU : 0.692 ± 0.238